

Pillar A des REALOQS-Formalismus

Strikte Ableitungsketten, Stage 1/Stage 2, Exportflächen und historische Pfade

Codebasis: REALOQS_v0.0604

Editor: Jan Seeck (antaris)

Ghostwriter: ChatGPT

24. März 2026

Inhaltsverzeichnis

1	Ziel, Geltungsbereich und Lesart	5
2	Architekturüberblick	5
2.1	Primitive Daten und abgeleitete Daten	5
2.2	Aktive, historische und Scaffold-Module	5
3	Primitive Policy und deterministische abgeleitete Parameter	7
3.1	Policy	7
4	IDEAL-Substrat und Tree-of-Cliques-Grundkonstruktion	9
4.1	Adresssubstrat	9
4.2	ToC-Vertex-, Boundary- und Interior-Typen	9
5	Adjazenz, Leitfähigkeit und Laplace-Operator	10
6	Generische OQS-Split- und DtN-Schicht	11
6.1	Blockzerlegung	11
6.2	Nicht-stabiler DtN-Schur-Komplement-Schritt	11
7	Deterministische Stabilisierung des DtN-Schritts	12
8	Generische Zwei-Stufen-Schnittstelle: Stage 1 und Stage 2	13
8.1	Abstrakte Pipeline	13
8.2	Stage 2 als Auswahl-/Subsystem-Schritt	14
9	Konkreter ToC-Träger: historischer nicht-stabiler Pfad	14
10	Konkreter ToC-Träger: aktiver stabiler Stage-1-Pfad	14
10.1	Der konkrete Zustandsraum	14
10.2	Der konkrete Stage-1-Schritt	15
10.3	End-to-end-Spezialisierung auf den kanonischen Laplace-Zustand	16
11	Semantische Identifikation von Stage 1	16
12	Stage 2 im konkreten ToC-Adapter	17

13 Strikte $A \rightarrow B$-Handoff-Fläche und Lean-Endobjekte	17
13.1 Allgemeine Handoff-Daten	17
13.2 Konkrete ToC-Exportobjekte	17
13.3 Weitere aus Pillar A exportierte diagnostische Größen	18
14 Weitere abgeleitete Lean-Endobjekte auf Basis von Pillar A	19
15 Belastbare Gesamtableitung des aktiven strikten Pfades	19
16 Schlussbemerkung	20
A Deklaratives Inventar der Pillar-A- und ToC-bezogenen Lean-Symbole	20
A.1 PillarA/Core/AQFTHandoff.lean	20
A.2 PillarA/Core/Approximant.lean	20
A.3 PillarA/Core/BInterfaces/ChannelNet.lean	21
A.4 PillarA/Core/BInterfaces/CovariantNets.lean	21
A.5 PillarA/Core/BInterfaces/GlobalCone.lean	21
A.6 PillarA/Core/BInterfaces/LocalAlgebraNet.lean	21
A.7 PillarA/Core/BInterfaces/RelativeEntropyHook.lean	22
A.8 PillarA/Core/BInterfaces/StateNet.lean	22
A.9 PillarA/Core/BoundaryPorts.lean	22
A.10 PillarA/Core/CylinderSets.lean	22
A.11 PillarA/Core/Derived/GNS.lean	22
A.12 PillarA/Core/Derived/GNS_CStar.lean	23
A.13 PillarA/Core/Derived/BridgeToSubstrate.lean	23
A.14 PillarA/Core/Derived/ModularKMS.lean	23
A.15 PillarA/Core/Determinism.lean	23
A.16 PillarA/Core/DirichletLaplacian.lean	23
A.17 PillarA/Core/DirichletLaplacianBridge.lean	24
A.18 PillarA/Core/DynamicsEquivariant.lean	24
A.19 PillarA/Core/Gates.lean	24
A.20 PillarA/Core/InfiniteCarrier.lean	25
A.21 PillarA/Core/InfiniteCarrierSymmetry.lean	25
A.22 PillarA/Core/MatrixNorms.lean	25
A.23 PillarA/Core/NoetherHooks.lean	26
A.24 PillarA/Core/Policy.lean	26
A.25 PillarA/Core/RegionCore.lean	27
A.26 PillarA/Core/RegionNet.lean	27
A.27 PillarA/Core/ResistanceMetric.lean	27
A.28 PillarA/Core/ResistanceMetricCanonical.lean	28
A.29 PillarA/Core/Selection.lean	28
A.30 PillarA/Core/SubstrateClass.lean	28
A.31 PillarA/Core/SymmetryAction.lean	29
A.32 PillarA/Core/SymmetryCoverageGates.lean	29
A.33 PillarA/Core/TailEliminationCoherence.lean	29
A.34 PillarA/Core/TailInterface.lean	30
A.35 PillarA/Core/ToC/AQFTSubstrate.lean	30
A.36 PillarA/Core/VacuumOffsetHook.lean	30
A.37 PillarA/Geometry/DimensionHooks.lean	30

A.38 PillarA/Ideal/Adapter/InfiniteCarrierInstance.lean	31
A.39 PillarA/Ideal/Adapter/RegionNetEmbedding.lean	31
A.40 PillarA/Ideal/Adapter/Seed.lean	31
A.41 PillarA/Ideal/Adapter/SubstrateInstance.lean	31
A.42 PillarA/Ideal/Adapter/TreeOfCliquesAQFTHandoff.lean	31
A.43 PillarA/Ideal/Adapter/TreeOfCliquesAllCellsConsistency.lean	32
A.44 PillarA/Ideal/Adapter/TreeOfCliquesApprox.lean	32
A.45 PillarA/Ideal/Adapter/TreeOfCliquesApproxDtNStabilized.lean	33
A.46 PillarA/Ideal/Adapter/TreeOfCliquesDtNSemantics.lean	34
A.47 PillarA/Ideal/Adapter/TreeOfCliquesDtNStabilizedSemantics.lean	34
A.48 PillarA/Ideal/Foliation.lean	35
A.49 PillarA/Ideal/IdealSymmetryNoether.lean	35
A.50 PillarA/Ideal/LCPMeasure.lean	35
A.51 PillarA/Ideal/SpacetimePaths.lean	36
A.52 PillarA/Ideal/Substrate.lean	36
A.53 PillarA/Ideal/TreeOfCliques/Boundary.lean	37
A.54 PillarA/Ideal/TreeOfCliques/CoarsenBoundary.lean	37
A.55 PillarA/Ideal/TreeOfCliques/DtN.lean	37
A.56 PillarA/Ideal/TreeOfCliques/DtNStabilized.lean	37
A.57 PillarA/Ideal/TreeOfCliques/DtNStabilizedDefs.lean	38
A.58 PillarA/Ideal/TreeOfCliques/DtNStabilizedGate.lean	38
A.59 PillarA/Ideal/TreeOfCliques/Edges.lean	38
A.60 PillarA/Ideal/TreeOfCliques/Laplacian.lean	38
A.61 PillarA/Ideal/TreeOfCliques/Level1Facts.lean	39
A.62 PillarA/Ideal/TreeOfCliques/SmallScales.lean	39
A.63 PillarA/Ideal/TreeOfCliques/Split.lean	39
A.64 PillarA/Ideal/TreeOfCliques/Vertex.lean	39
A.65 PillarA/Kinematics/Causality.lean	40
A.66 PillarA/Kinematics/Clock.lean	40
A.67 PillarA/Kinematics/DiscreteSpacetime.lean	40
A.68 PillarA/Kinematics/FrameChange.lean	41
A.69 PillarA/Kinematics/FromUpdate.lean	41
A.70 PillarA/Kinematics/KinematicsHooks.lean	41
A.71 PillarA/Kinematics/ProperTime.lean	41
A.72 PillarA/Kinematics/ProperTimeFromMetric.lean	42
A.73 PillarA/Kinematics/SpacetimeRegions.lean	42
A.74 PillarA/Kinematics/Worldline.lean	42
A.75 PillarA/QQS/DtN.lean	42
A.76 PillarA/QQS/DtNStabilized.lean	43
A.77 PillarA/QQS/DtNStabilizedCriteria.lean	43
A.78 PillarA/QQS/DtN_TailHook.lean	43
A.79 PillarA/QQS/LiebRobinsonHook.lean	43
A.80 PillarA/QQS/SplitInvariants.lean	44
A.81 PillarA/QQS/SysEnv.lean	44
A.82 PillarA/Physics/ScaleBreakingMismatchD.lean	44
A.83 PillarA/Physics/Instances/ScaleAxisTreeOfCliquesDtN.lean	45
A.84 PillarA/Physics/Instances/TreeOfCliquesAxisCanonicalD.lean	45
A.85 PillarA/Physics/Instances/TreeOfCliquesAxisCanonicalLawsD.lean	45

A.86	PillarA/Physics/Instances/TreeOfCliquesBoundaryK1Shape.lean	45
A.87	PillarA/Physics/Instances/TreeOfCliquesBreakWitness.lean	46
A.88	PillarA/Physics/ScaleBreaking.lean	46
A.89	PillarA/Physics/ScaleBreakingFaithfulness.lean	47
A.90	PillarA/Physics/ScaleBreakingFaithfulnessD.lean	47
A.91	PillarA/Physics/ScaleBreakingMismatch.lean	47
A.92	PillarA/RG/ScaleWindow.lean	47
A.93	PillarA/Symmetry/ApproxIsometry.lean	48
A.94	PillarA/Symmetry/ApproxPoincare.lean	48
A.95	PillarA/Symmetry/PoincareLorentzSplit.lean	48
A.96	PillarA/Update/ApproximationPipeline.lean	49
A.97	PillarA/Update/DeterministicExport.lean	50
A.98	PillarA/Update/MismatchDriver.lean	50
A.99	PillarA/Update/MismatchFunctional/NormDiff.lean	50
A.100	PillarA/Update/MismatchFunctional.lean	50
A.101	PillarA/Update/OpObserver.lean	51
A.102	PillarA/Update/Seed.lean	51
A.103	PillarA/Update/Step.lean	51
A.104	PillarA/Update/TailEliminationDtN.lean	52
A.105	PillarA/Update/TailEliminationDtNSTabilized.lean	52
A.106	Examples/TreeOfCliques_AQFT_Derived.lean	52
A.107	Examples/TreeOfCliques_AQFT_FromA.lean	52
A.108	Examples/TreeOfCliques_AQFT_HK_Instances.lean	52
A.109	Examples/TreeOfCliques_EndToEnd.lean	53
A.110	Examples/TreeOfCliques_EndToEnd_StabilizedConcrete.lean	53
A.111	Examples/TreeOfCliques_MatterFromA.lean	53
A.112	Examples/TreeOfCliques_OQS_CompilerFromA.lean	54
A.113	Examples/TreeOfCliques_OQS_FromA.lean	54
A.114	Examples/TreeOfCliques_OQS_FromCoarsen.lean	54
A.115	Examples/TreeOfCliques_RealTower_FromDtNSTabilized.lean	55
A.116	Examples/TreeOfCliques_ScaleAxisHarness.lean	55
A.117	Nicht separat inventarisierte Import-/Umbrella-Module	55

1 Ziel, Geltungsbereich und Lesart

Dieses Dokument ist eine code-treue, mathematisch formulierte Dokumentation von *Pillar A* der bereitgestellten Lean-Codebasis `REALOQS_v0.0604`. Ziel ist, dass ein Reviewer den formalen Gehalt des derzeitigen aktiven Pillar-A-Pfades aus dem Dokument selbst verstehen kann, ohne fortlaufend in den Lean-Code springen zu müssen. Gleichzeitig werden historische, ältere oder nicht mehr aktive Pfade ausdrücklich markiert, damit nicht versehentlich ein Legacy-Pfad mit dem aktuellen strikten Kernpfad verwechselt wird.

Inhaltlich trennt das Dokument vier Ebenen:

1. den **strikten aktiven Kernpfad** von der ToC-Konstruktion bis zur $A \rightarrow B$ -Exportfläche,
2. die **abstrakten generischen Oberflächen** von Stage 1 und Stage 2,
3. die **historischen bzw. Legacy-Pfade** (insbesondere der nicht-stabilisierte DtN-Pfad),
4. die **abgeleiteten Lean-Endobjekte** und Exportobjekte, die aus Pillar A hervorgehen.

Wichtige Einschränkung. “Vollständig” bedeutet hier: der strikte aktive Formalismus wird mit Definitionen, Aussagen und Beweisideen mathematisch ausgeschrieben; zusätzlich gibt ein Appendix eine deklarative Inventarliste der in `PillarA` und den ToC-bezogenen Beispieldateien vorkommenden Lean-Symbole (`def`, `abbrev`, `structure`, `theorem`, `lemma`, `class`, `inductive`). Dadurch ist der strukturelle Umfang des Codes dokumentiert, auch wenn nicht jede Hilfslemma-Zeile in natürlichsprachiger Vollprosa erneut ausgeschrieben wird.

2 Architekturüberblick

2.1 Primitive Daten und abgeleitete Daten

Der aktive strikte Pfad von Pillar A hat als primitive Eingaben im Wesentlichen nur:

$$b \in \mathbb{N}, \quad p : \text{Policy}(b), \quad p.L_{\max} \in \mathbb{N}. \quad (1)$$

Dabei ist b der Verzweigungsfaktor des Adressalphabets und in Lean ein *Typparameter* der Struktur $\text{Policy}(b)$, nicht ein Feld von p . Das einzige primitive Feld von p auf dem strikten Pfad ist $L_{\max}$ als Trunktationsregulator. Alle weiteren Größen auf dem aktiven strikten Pfad werden kanonisch aus $(b, p.L_{\max})$ abgeleitet.

2.2 Aktive, historische und Scaffold-Module

Zentrale Auswahl des aktiven strikten Kernpfads. Die folgende Tabelle ist bewusst eine *Auswahl der zentralen Module* des aktiven heutigen Kernpfads; das vollständige statusmarkierte Modulbild steht im Appendix. In der Auswahl sind sowohl die fachlich sichtbaren ToC-/DtN-Module als auch einige fundamentale Trägermodule aufgenommen, ohne damit bereits jeden strikt inventarisierten Baustein zu wiederholen.

Modul	Status
<code>PillarA/Core/Policy.lean</code>	aktiv/strikt
<code>PillarA/Core/Gates.lean</code>	aktiv/strikt
<code>PillarA/Core/DirichletLaplacian.lean</code>	aktiv/strikt
<code>PillarA/Core/Determinism.lean</code>	aktiv/strikt

Modul	Status
PillarA/Core/MatrixNorms.lean	aktiv/strikt
PillarA/Core/RegionNet.lean	aktiv/strikt
PillarA/Core/SubstrateClass.lean	aktiv/strikt
PillarA/Ideal/Substrate.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/Vertex.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/Boundary.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/Edges.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/Laplacian.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/Split.lean	aktiv/strikt
PillarA/OQS/SysEnv.lean	aktiv/strikt
PillarA/OQS/DtN.lean	aktiv/strikt
PillarA/OQS/DtNStabilized.lean	aktiv/strikt
PillarA/OQS/DtNStabilizedCriteria.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/DtNStabilized.lean	aktiv/strikt (Umbrella-Re-Export)
PillarA/Ideal/TreeOfCliques/DtNStabilizedDefs.lean	aktiv/strikt
PillarA/Ideal/TreeOfCliques/DtNStabilizedGate.lean	aktiv/strikt
PillarA/Update/ApproximationPipeline.lean	aktiv/strikt
PillarA/Update/TailEliminationDtNStabilized.lean	aktiv/strikt
PillarA/Ideal/Adapter/TreeOfCliquesApproxDtNStabilized.lean	aktiv/strikt
PillarA/Ideal/Adapter/TreeOfCliquesDtNStabilizedSemantics.lean	aktiv/strikt
PillarA/Ideal/Adapter/TreeOfCliquesAllCellsConsistency.lean	aktiv/strikt
PillarA/Ideal/Adapter/TreeOfCliquesAQFTHandoff.lean	aktiv/strikt
PillarA/Core/AQFTHandoff.lean	aktiv/strikt
PillarA/Physics/Instances/TreeOfCliquesAxisCanonicalD.lean	aktiv/strikt
PillarA/Physics/Instances/TreeOfCliquesBreakWitness.lean	aktiv/strikt

Historische oder nicht mehr aktive Pfade.

Modul	Status
PillarA/Update/TailEliminationDtN.lean	historisch bzw. nicht der aktuelle aktive Kernpfad
PillarA/Ideal/TreeOfCliques/DtN.lean	historisch bzw. nicht der aktuelle aktive Kernpfad
PillarA/Ideal/Adapter/TreeOfCliquesApprox.lean	historisch bzw. nicht der aktuelle aktive Kernpfad
PillarA/Ideal/Adapter/TreeOfCliquesDtNSemantics.lean	historisch bzw. nicht der aktuelle aktive Kernpfad
PillarA/Update/TwoStageApprox.lean	historisch bzw. nicht der aktuelle aktive Kernpfad

Bemerkung zum Umbrella-Modul. `PillarA/Ideal/TreeOfCliques/DtNStabilized.lean` enthält selbst keine neuen Definitionen, sondern re-exportiert explizit `DtNStabilizedDefs.lean` und `DtNStabilizedGate.lean`. Genau deshalb gehört es zum aktiven strikten Pfad: ohne diesen Umbrella-Import fehlen die Gate-Sätze für den stabilisierten DtN-Abschluss im Scope.

Retainte Scaffold-/Hook- und Nebenpfade.

Modul/Direktorium	Einordnung
<code>PillarA/Geometry</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/Kinematics</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/Symmetry</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/Scaffold</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/Matter</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/RG</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades
<code>PillarA/Core/BInterfaces</code>	Hook-, Interface- oder Scaffold-Schicht; nicht Teil des derzeitigen strikten Stage- 1 $\rightarrow$ A/B-Kernpfades

3 Primitive Policy und deterministische abgeleitete Parameter

3.1 Policy

Die Lean-Struktur `PillarA.LayerA.Policy` ist eine Familie $\text{Policy}(b)$ über dem festen Verzweigungsparameter b und reduziert den primitiven Policy-Inhalt auf den Trunkationsregulator:

$$p : \text{Policy}(b), \quad p.L_{\max} \in \mathbb{N}. \quad (2)$$

Das heißt: b ist ein externer Typparameter, während $L_{\max}$ das einzige primitive Feld von p ist. Weitere Felder sind nicht primitiv, sondern abgeleitet. Formal werden insbesondere definiert:

$$L_{\text{int}}(p) := 1, \quad (3)$$

$$K_{\max}(p) := \text{pred}(L_{\max}), \quad (4)$$

$$r_*(p) := \frac{1}{b+1}, \quad (5)$$

$$\eta(p) := 1. \quad (6)$$

Zum sichtbaren Policy-Namensraum gehört außerdem der historische Tag-Typ `GroundRule` = `{minId,minLex}`. Auf dem aktuellen strikten Pfad wird er nicht frei gewählt, sondern deterministisch durch

$$\text{groundRule}(p) := \text{minId} \quad (7)$$

fixiert; die Tags bleiben nur aus Dokumentations- und Kompatibilitätsgründen im Code sichtbar. Außerdem wird ein deterministischer numerischer Regularisierungsmaßstab aus der Float64-Maschinengenauigkeit in mathematischer Form abgeleitet:

$$\varepsilon_{\text{mach}} := 2^{-52}, \quad (8)$$

$$\varepsilon_{\text{floor}} := \sqrt{\varepsilon_{\text{mach}}}, \quad (9)$$

$$\varepsilon(p) := \varepsilon_{\text{floor}}, \quad (10)$$

$$\varepsilon_0(p) := \max(r_*(p)^{L_{\text{max}}}, \varepsilon_{\text{floor}}). \quad (11)$$

Ferner werden normierte Skalengewichte α_k über

$$\alpha_{\text{Denom}}(p) := \sum_{j=1}^{K_{\text{max}}(p)} r_*(p)^j \quad (12)$$

und

$$\alpha_k(p) := \begin{cases} \frac{r_*(p)^k}{\alpha_{\text{Denom}}(p)}, & k \in \{1, \dots, K_{\text{max}}(p)\}, \\ 0, & \text{sonst,} \end{cases} \quad (13)$$

definiert. Wichtig ist die Lean-Konvention: Falls $\alpha_{\text{Denom}}(p) = 0$ sein sollte, wird dennoch per Felddivision ausgewertet; Normalisierungs- oder Positivitätseigenschaften der Gewichte werden dann nicht implizit angenommen, sondern nur separat unter den jeweils passenden Voraussetzungen bewiesen.

Degenerierte Regime. Die beiden im Code sichtbaren Grenzregime sind reviewerrelevant:

- (i) Für $b = 0$ ist $\text{Fin}(0)$ leer, also gibt es keine echten Kinderadressen. Dann ist $r_*(p) = 1$, positive Ebenen enthalten keine Adressen, und für $L_{\text{max}} \geq 1$ wird der Boundary-Träger $\Omega = B_{L_{\text{max}}}$ leer.
- (ii) Für $L_{\text{max}} \leq 1$ gilt $K_{\text{max}}(p) = 0$. Damit ist die Summe $\alpha_{\text{Denom}}(p) = \sum_{j=1}^{K_{\text{max}}(p)} r_*(p)^j$ leer und also gleich 0; die gesamte α -Gewichtung degeneriert in diesem Regime zur trivialen Konvention.

Diese Fälle sind im Code nicht verboten, sondern werden als definierte, aber strukturell degenerierte Randregime behandelt. Aussagen wie Nichtleerheit von Ω oder nichttriviale Scale-Breaking-Diagnostik werden deshalb separat unter passenden Positivitätsannahmen an b bzw. L_{max} formuliert.

Theorem 3.1 (Stabilität der abgeleiteten Policy-Daten). *Sind $p = q$, so stimmen alle aus p deterministisch abgeleiteten Größen wie K_{max} , ε_0 , α_{Denom} und sämtliche α_k überein.*

Beweis. Dies ist genau der Gehalt von `gate_POLICY_derived_stable` in `PillarA/Core/Policy.lean`. Der Beweis ist rein definitional: nach $p = q$ reduzieren alle abgeleiteten Terme durch Auswertung ihrer Definitionen auf dieselben Ausdrücke. $\square$

4 IDEAL-Substrat und Tree-of-Cliques-Grundkonstruktion

4.1 Adresssubstrat

Das IDEAL-Substrat modelliert Zellenadressen als endliche Wörter über dem Alphabet $\text{Fin}(b)$:

$$\text{Addr}(b) := \text{List}(\text{Fin}(b)). \quad (14)$$

Zu einer Adresse a gehören ihre Ebene $\text{level}(a) = |a|$, ihr Kind $\text{child}(a, d) = a ++ [d]$ und die Menge aller Kinder

$$\text{children}(a) := \{a ++ [d] : d \in \text{Fin}(b)\}. \quad (15)$$

Auf dieser Grundlage definiert der Code die diskreten Schalen

$$\text{cellsAtLevel}(b, k) := \{a \in \text{Addr}(b) : |a| = k\} \quad (16)$$

und die endlichen Trunkationen

$$\text{cellsUpTo}(b, 0) := \text{cellsAtLevel}(b, 0), \quad \text{cellsUpTo}(b, L+1) := \text{cellsUpTo}(b, L) \cup \text{cellsAtLevel}(b, L+1). \quad (17)$$

Lemma 4.1 (Charakterisierung fester Ebenen). *Für jede Adresse a gilt*

$$a \in \text{cellsAtLevel}(b, k) \iff |a| = k. \quad (18)$$

Beweis. Dies ist `mem_cellsAtLevel_iff_length`. Die Hinrichtung liest a als Bild eines Wortes der Länge k , die Rückrichtung konstruiert aus einer Liste der Länge k eine Funktion $\text{Fin}(k) \rightarrow \text{Fin}(b)$, deren `List.ofFn` wieder a ergibt. $\square$

4.2 ToC-Vertex-, Boundary- und Interior-Typen

Für ein fixes Trunkationsniveau k werden definiert:

$$V_k := \text{Vertex}(b, k) = \text{cellsUpTo}(b, k), \quad (19)$$

$$B_k := \text{Boundary}(b, k) = \text{cellsAtLevel}(b, k), \quad (20)$$

$$I_k := \text{Interior}(b, k) = V_k \setminus B_k. \quad (21)$$

Alle drei Mengen werden als endliche Lean-Typen realisiert.

Lemma 4.2 (Tiefenabschätzung). *Für jedes $v \in V_k$ gilt $\text{depth}(v) \leq k$.*

Beweis. Die Aussage ist `Vertex.depth_le` und folgt unmittelbar aus der Definition von V_k als `cellsUpTo(b, k)`. $\square$

Proposition 4.3 (Kanonescher Boundary/Interior-Split). *Es existiert eine kanonische Äquivalenz*

$$e_k : V_k \xrightarrow{\sim} B_k \sqcup I_k, \quad (22)$$

indem man einen Vertex nach der Bedingung $|a| = k$ als Boundary- oder Interior-Element klassifiziert.

Beweis. Dies ist `Split.equivVertexSum` bzw. verpackt `TreeOfCliques.split`. Die Vorwärtsabbildung `toSum` entscheidet per Fallunterscheidung $|a| = k$ gegen $|a| \neq k$; die Rückabbildung `fromSum` ist einfach die jeweilige Inklusion in V_k . Beide Inversen werden im Code durch explizite Fallunterscheidung bewiesen. $\square$

5 Adjazenz, Leitfähigkeit und Laplace-Operator

Die rohe Adjazenz auf Adressen ist die Vereinigung zweier Relationen:

- (a) Parent-Kanten zwischen Eltern- und Kindadresse,
- (b) Sibling-Kanten zwischen verschiedenen Adressen gleicher Länge mit demselben Vorgänger `dropLast`.

Daraus wird `adjAddr` und auf V_k die adjungierte Relation `Adj` erzeugt. Die kanonische Leitfähigkeitsfunktion ist

$$c_k(x, y) := \begin{cases} 1, & \text{falls } \text{Adj}(x, y), \\ 0, & \text{sonst.} \end{cases} \quad (23)$$

Lemma 5.1 (Symmetrie und Nichtnegativität der Leitfähigkeit). *Für alle $x, y \in V_k$ gilt*

$$c_k(x, y) = c_k(y, x), \quad c_k(x, y) \geq 0. \quad (24)$$

Beweis. Dies sind `conductance_symm` und `conductance_nonneg`. Die Symmetrie folgt aus `Adj_symm`, die wiederum aus der Symmetrie von Parent- und Sibling-Relationen folgt. Die Nichtnegativität ist wegen der Werte 0 oder 1 trivial. $\square$

Die in `PillarA/Core/Gates.lean` bereitgestellten generischen Layer-A-Grundobjekte lauten auf einem endlichen Träger V :

$$\text{Weight}(V) := V \rightarrow V \rightarrow \mathbb{R}, \quad (25)$$

$$\text{deg}_c(i) := \sum_{j \in V} c(i, j), \quad (26)$$

$$(\Delta c)(i, j) := \begin{cases} \text{deg}_c(i), & i = j, \\ -c(i, j), & i \neq j, \end{cases} \quad (27)$$

$$\text{AdmissibleOp}(\Lambda) := (\Lambda^\top = \Lambda) \wedge (\forall i, \sum_j \Lambda_{ij} = 0) \wedge (\forall i \neq j, \Lambda_{ij} \leq 0). \quad (28)$$

Damit ist der ToC-Laplacian nicht bloß eine informelle Matrix, sondern ein Spezialfall der generischen Lean-Funktion `laplacian`, und das spätere Admissibilitätsresultat ist genau die Aussage, dass $\Delta(c_k)$ diese drei Bedingungen erfüllt.

Der ToC-Laplacian wird dann als kanonischer Laplace-Operator der Leitfähigkeit definiert:

$$L_k := \Delta(c_k). \quad (29)$$

Theorem 5.2 (Admissibilität des ToC-Laplacians). *Der so gebildete Operator L_k erfüllt die generischen Laplace-Admissibilitätsbedingungen der Layer-A-Basissschicht.*

Beweis. Dies ist `gate_LAPLACIAN_ADMISSIBLE_treeOfCliques`. Die Voraussetzungen des generischen Gates `gate_LAPLACIAN_ADMISSIBLE` werden durch die eben bewiesene Symmetrie und Nichtnegativität der Leitfähigkeit erfüllt. $\square$

Für die spätere PSD-Kette benutzt `PillarA/Core/DirichletLaplacian.lean` die expliziten

Grundgrößen

$$c^{\text{off}}(i, j) := \begin{cases} 0, & i = j, \\ c(i, j), & i \neq j, \end{cases} \quad (30)$$

$$\mathcal{E}_c(f) := \frac{1}{2} \sum_{i,j} c^{\text{off}}(i, j) (f(i) - f(j))^2, \quad (31)$$

$$\text{quadForm}(\Lambda, f) := \sum_i f(i) (\Lambda \cdot f)_i. \quad (32)$$

Formal sind dies `offDiag`, `dirichletEnergy` und `quadForm`. Der zentrale Brückensatz ist

$$\mathcal{E}_c(f) = \text{quadForm}(\Delta c, f), \quad (33)$$

also `gate_DIRICHLET_EQ_LAPLACIAN_QUAD`; zusammen mit $\mathcal{E}_c(f) \geq 0$ bei $c \geq 0$, formal `gate_DIRICHLET_NONNEG`, liefert dies genau den Quadratform-Eingang für die spätere PSD- und Determinantenkette.

6 Generische OQS-Split- und DtN-Schicht

6.1 Blockzerlegung

Die generische Struktur `PillarA.OQS.Split` kodiert eine endliche Zerlegung

$$V \cong B \sqcup I \quad (34)$$

und erlaubt für jede Matrix $L \in \mathbb{R}^{V \times V}$ die Blockextraktion

$$L_{BB}, \quad L_{BI}, \quad L_{IB}, \quad L_{II}. \quad (35)$$

Wesentliche Hilfssätze der Split-Schicht sind insbesondere die Transpositionssätze `blockBB_transpose_of_symm` und `blockII_transpose_of_symm`.

6.2 Nicht-stablisierter DtN-Schur-Komplement-Schritt

Sei L eine volle reelle Matrix auf V und sei w ein expliziter Beweisdatensatz dafür, dass L_{II} invertierbar ist. Die Struktur `InteriorInv` speichert genau einen Kandidaten L_{II}^{-1} samt beidseitiger Inverseneigenschaft. Der generische DtN-Operator ist dann

$$\text{DtN}(L) := L_{BB} - L_{BI} L_{II}^{-1} L_{IB}. \quad (36)$$

Zusätzlich definiert der Code eine innere Eliminationslösung

$$u_I := -L_{II}^{-1} L_{IB} \varphi \quad (37)$$

und den daraus induzierten Randfluss.

Theorem 6.1 (Randfluss = DtN-Multiplikation). *Für jede Randpotenzialfunktion φ gilt*

$$\text{boundaryFlux}(L, \varphi, u_I) = \text{DtN}(L) \varphi. \quad (38)$$

Beweis. Dies ist `boundaryFlux_eq_DtN_mulVec` in `PillarA/OQS/DtN.lean`. Der Beweis ist reine Blockalgebra: der Term $u_I = -L_{II}^{-1} L_{IB} \varphi$ wird in den Randfluss eingesetzt und die Matrixprodukte

werden zu genau dem Schur-Komplement zusammengefasst. Bemerkenswert ist, dass für diese Identität die Tatsache, dass w ein echter Inversenbeweis ist, gar nicht gebraucht wird; benötigt wird sie erst bei Aussagen, die tatsächliche Well-posedness verwenden. $\square$

7 Deterministische Stabilisierung des DtN-Schritts

Der aktive strikte Pfad verwendet nicht den rohen DtN-Schritt, sondern den stabilisierten Pfad aus `PillarA/OQS/DtNStabilized.lean`. Für eine Quadratmatrix A wird zunächst symmetrisiert:

$$\text{symm}(A) := \frac{1}{2}(A + A^\top). \quad (39)$$

Dabei ist die verwendete Frobenius-Norm explizit

$$\|A\|_F := \sqrt{\sum_{i,j} A_{ij}^2}, \quad (40)$$

formal `MatrixNorm.frob`. Dann wird mit einer rein numerischen, aus $p.\varepsilon$ abgeleiteten Jittergröße stabilisiert:

$$\delta(A) := \varepsilon(p) \max(1, \|A\|_F), \quad (41)$$

formal `deltaSPD`, und

$$\text{stabilize}(A, \delta) := A + \delta I. \quad (42)$$

Für den Interior-Block entsteht somit

$$L_{II}^{\text{stab}} := \text{symm}(L_{II}) + \delta(\text{symm}(L_{II}))I. \quad (43)$$

Lemma 7.1 (Positivität der numerischen Stabilisierung). *Für jede zulässige Matrix A gilt*

$$\delta(A) > 0, \quad \delta(A) \geq \varepsilon(p). \quad (44)$$

Beweis. Dies sind `deltaNum_pos` und `deltaNum_ge_eps`. Der Beweis benutzt nur $\varepsilon(p) > 0$ und $\max(1, \|A\|_F) \geq 1$. $\square$

Das generische PSD $\Rightarrow$ Det-Kriterium ist in `PillarA/OQS/DtNStabilizedCriteria.lean` separat formalisiert:

$$A \text{ positiv semidefinit, } \delta > 0 \implies \det(\text{stabilize}(A, \delta)) \neq 0, \quad (45)$$

formal `det_stabilize_ne_zero_of_posSemidef`. Dessen direkte Interior-Block-Spezialisierung `det_LII_stabilized_ne_zero_of_posSemidef` ist genau die generische mathematische Schablone, die später im ToC-Fall mit dem PSD-Gate des Laplacian-Blocks zusammengesetzt wird.

Der stabilisierte DtN-Operator erscheint im Code in zwei Varianten. Zunächst gibt es die zeugenlose Definition

$$\text{DtN}_{\text{stab}}(L) := L_{BB} - L_{BI}(L_{II}^{\text{stab}})^{-1}L_{IB}, \quad (46)$$

formal `DtN_stabilized`. Sie benutzt direkt die kanonische Mathlib-Inversion von L_{II}^{stab} , enthält aber noch keine Well-posedness-Aussage. Daneben gibt es die zeugenbasierte Variante

$$\text{DtN}_{\text{stab}}^{\text{of}}(L, w) := L_{BB} - L_{BI} w L_{IB}, \quad (47)$$

formal `DtN_stabilized_of`, wobei w der in `InteriorInvStabilized` gespeicherte Inversezeuge

für L_{II}^{stab} ist. Aus einem Det-Gate

$$\det(L_{II}^{\text{stab}}) \neq 0 \quad (48)$$

konstruiert `interiorInvStabilized_of_det` deterministisch den Zeugen $(L_{II}^{\text{stab}})^{-1}$, und `dtn_stabilized_of_det` ist der bequeme Det-Gate-zu-DtN-Konstruktor entlang dieses zeugenbasierten Pfads.

8 Generische Zwei-Stufen-Schnittstelle: Stage 1 und Stage 2

8.1 Abstrakte Pipeline

Die Struktur `TwoStageApprox` abstrahiert die Update-Architektur von Pillar A. Sie besteht nicht nur aus Operatoren auf einem Zustandsraum, sondern aus einem gesamten Vertragspaket mit Daten-, Typeclass- und Beweisfeldern:

$$(p, \text{Cell}, \text{instSubstrate}, S, \text{truncateTail}, \text{integrateTail}, \quad (49)$$

$$\text{select}, \text{truncate_idem}, \text{integrate_idem}, \text{stage1_idem}). \quad (50)$$

wobei

$$p : \text{Policy}(b), \quad (51)$$

$$\text{Cell} : \mathbb{N} \rightarrow \text{Type}, \quad (52)$$

$$\text{instSubstrate} : \text{Substrate}(\text{Cell}) \text{ als Pflicht-Typeclass}, \quad (53)$$

$$S \text{ ein Zustandsraum}, \quad (54)$$

$$\text{select} : \text{SubsystemMode}(\text{Cell}(p.L_{\max})) \rightarrow S \rightarrow S. \quad (55)$$

Die zulässigen Auswahlmodi sind auf einem endlichen Indexträger α gegeben durch

$$\text{SubsystemMode}(\alpha) = \text{region}(R) \text{ mit } R : \text{Finset}(\alpha) \text{ oder } \text{allCells}. \quad (56)$$

Insbesondere parametrisieren `policy` und `Cell` den Typ der feinsten Zellen $\text{Cell}(p.L_{\max})$, auf dem Stage 2 operiert; diese Daten sind also Teil der Struktur und nicht bloß impliziter Kontext. Stage 1 und die volle Pipeline werden definitorisch gesetzt als

$$\text{stage1}(s) := \text{integrateTail}(\text{truncateTail}(s)), \quad (57)$$

$$\text{pipeline}(m, s) := \text{select}(m, \text{stage1}(s)). \quad (58)$$

Theorem 8.1 (Pflichtfelder und Grundgesetze von Stage 1). *Die Schnittstelle fordert als Strukturfelder:*

$$\text{truncateTail} \circ \text{truncateTail} = \text{truncateTail}, \quad (59)$$

$$\text{integrateTail} \circ \text{integrateTail} = \text{integrateTail}, \quad (60)$$

$$\text{stage1} \circ \text{stage1} = \text{stage1}. \quad (61)$$

Beweis. Diese Gesetze sind keine extern nachträglich bewiesenen Meta-Sätze, sondern echte Pflichtfelder der Lean-Struktur `TwoStageApprox`. Die zugehörigen Gate-Sätze `gate_TRUNCATE_IDEMPOTENT`, `gate_INTEGRATE_IDEMPOTENT` und `gate_STAGE1_IDEMPOTENT` exponieren genau diese bereits in der Struktur gespeicherten Beweise nochmals als Theoreme. $\square$

8.2 Stage 2 als Auswahl-/Subsystem-Schritt

Stage 2 ist *formal vorhanden*, aber im aktuell aktiven Pillar-A $\rightarrow$ Pillar-B-Kernpfad nicht der Schritt, der für Pillar B genutzt wird. Formal ist Stage 2 die **select**-Operation auf `SubsystemMode`. Dazu existieren in der generischen Pipeline drei Kontraktflächen:

- (i) **Monotonie** über `MonotoneLaws`,
- (ii) **all-cells vs. single-cell consistency** über `AllCellsConsistency`,
- (iii) **Äquivarianz** über `Equivariance.Laws`.

Im strikten A $\rightarrow$ B-Pfad wird derzeit nur Stage 1 aktiv für die Exportfläche verwendet; Stage 2 bleibt als formalisierte, aber für Pillar B noch nicht aktive Subsystem-/Region-Selektion erhalten.

9 Konkreter ToC-Träger: historischer nicht-stabilisierter Pfad

Der ältere konkrete Adapter `TreeOfCliquesApprox` speichert im Zustand die volle Blockstruktur

$$(L_{BB}, L_{BI}, L_{IB}, L_{II}, L_{II}^{-1}). \quad (62)$$

Die Stage-1-Reduktion setzt dort

$$L_{BB} \mapsto \text{DtN}(L), \quad L_{BI} \mapsto 0, \quad L_{IB} \mapsto 0, \quad (63)$$

und belässt L_{II} sowie dessen Inversenzeugen als entkoppelte Hilfsdaten. Die zugehörigen Schlüsselaussagen sind:

- `dtn_fullMatrix_of_cross_zero`,
- `dtn_fullMatrix_of_reduced`,
- `reduce_idem`,
- der Semantik-Adapter `TreeOfCliquesDtNSemantics.mk`.

Dieser Pfad ist historisch und dokumentationsrelevant, aber *nicht* der heute aktive strikte Kernpfad.

10 Konkreter ToC-Träger: aktiver stabilisierter Stage-1-Pfad

10.1 Der konkrete Zustandsraum

Der aktive Pfad `TreeOfCliquesApproxDtNStabilized` nimmt als kanonische Quellmatrix den ToC-Laplacian auf der Endskala

$$L_0 := L_k, \quad k := p \cdot L_{\max}. \quad (64)$$

Aus diesem werden die vier Blockmatrizen $L_{BB}^0, L_{BI}^0, L_{IB}^0, L_{II}^0$ exponiert. Das Modul führt zudem ein starkes Gate ein:

$$h_{\text{psd},0} : \text{symm}(L_{II}^0) \text{ ist positiv semidefinit}, \quad (65)$$

$$h_{\text{det},0} : \det(L_{II}^{\text{stab},0}) \neq 0. \quad (66)$$

Die Lean-Datei beweist `quadForm_L0_nonneg` mittels der Dirichlet-Identität und der Nichtnegativität der Dirichlet-Energie. Für den eigentlichen PSD- und Determinantenabschluss wird jedoch explizit das aktive Gate-Modul `PillarA/Ideal/TreeOfCliques/DtNStabilizedGate.lean` benutzt: dort liefert `quadForm_L_nonneg` den globalen Quadratformschritt, `quadForm_ext_eq_quadForm_blockII` die Nullrand-Restriktion, `hpsd_laplacian_blockII` die PSD-Aussage für den Interior-Block und `detGate_proved` den kanonischen Determinantenabschluss. `TreeOfCliquesApproxDtNStabilized.lean` spezialisiert diese Kette auf die Endskala $k = p.L_{\max}$ und erhält so `hpsd0_proved`, dann `hdet0_of_hpsd0` und schließlich den instanziierten Gate-Zeugen `hdet0_inst`.

Theorem 10.1 (Starker Abschluss des Stabilisierungsgates). *Für den kanonischen ToC-Laplacian auf der Endskala ist der stabilisierte Interior-Block invertierbar:*

$$\det(L_{II}^{\text{stab},0}) \neq 0. \quad (67)$$

Beweis. Der code-treue Pfad zerfällt in einen generischen Gate-Schritt und dessen Endskalen-Spezialisierung:

$$\begin{aligned} \text{quadForm_L_nonneg} &\implies \text{quadForm_ext_eq_quadForm_blockII} \\ &\implies \text{hpsd_laplacian_blockII} \implies \text{detGate_proved} \end{aligned}$$

im Modul `DtNStabilizedGate.lean`, und daraus in `TreeOfCliquesApproxDtNStabilized.lean`

$$\begin{aligned} \text{quadForm_L0_nonneg} &\implies \text{hpsd0_proved} \\ &\implies \text{hdet0_of_hpsd0} \implies \text{hdet0_inst}. \end{aligned}$$

Mathematisch: aus der nichtnegativen Dirichlet-Energie des ToC-Laplacians folgt zuerst die positive Semidefinitheit des Interior-Blocks unter Nullrand-Erweiterung; die zusätzliche positive Stabilisierung δI macht daraus einen strikt positiven und damit invertierbaren Block. $\square$

10.2 Der konkrete Stage-1-Schritt

Der aktive reduzierte Zustand speichert

$$(L_{II}, L_{BB}, L_{BI}, L_{IB}, h_{\det}), \quad (68)$$

also insbesondere kein separates Inversenfeld mehr, sondern nur das Det-Gate des stabilisierten Interior-Blocks. Präzise hat das fünfte Feld die Lean-Form

$$h_{\det} : \text{let } A := \text{symm}(L_{II}) \text{ in } \det(\text{stabilize}(A, \delta(A))) \neq 0, \quad (69)$$

wobei $\delta(A) = \text{deltaSPD}(p, A)$. Das Gate bezieht sich also nicht auf den rohen gespeicherten Interior-Block, sondern ausdrücklich auf dessen *symmetrisierte und dann stabilisierte* Version. Der zusammengesetzte Volloperator `fullMatrix` rekonstruiert daraus wieder die volle Matrix auf V_k .

Die aktive Stage-1-Reduktion ist dann

$$\text{reduce}(st) := (L_{II}, \text{DtN}_{\text{stab}}^{\text{of}}(\text{fullMatrix}(st), \text{invStab}(st)), 0, 0, h_{\det}). \quad (70)$$

Hier ist `invStab(st)` genau der aus $h_{\det}$ deterministisch konstruierte Zeuge der stabilisierten Interior-Inversen; `reduce` verwendet also explizit die *zeugenbasierte* DtN-Variante und nicht die rohe kanonische Matrixinversion.

Die konkrete `TwoStageApprox`-Instanz `TreeOfCliquesApproxDtNStabilized.mk` setzt dabei

$$\text{truncateTail} := \text{id}, \quad \text{integrateTail} := \text{reduce}, \quad \text{select} := \text{select}. \quad (71)$$

Daher degeneriert die generische Definition $\text{stage1} = \text{integrateTail} \circ \text{truncateTail}$ im aktiven Adapter zu

$$\text{stage1} = \text{reduce} \circ \text{id} = \text{reduce}. \quad (72)$$

Lemma 10.2 (Nullkreuzblöcke nach Reduktion). *Nach `reduce` gelten stets*

$$L_{BI} = 0, \quad L_{IB} = 0. \quad (73)$$

Beweis. Dies ist definitorisch in `reduce` eingebaut und wird etwa durch `reduce_cross_zero_stLap` exponiert. $\square$

Proposition 10.3 (Idempotenz der aktiven Stage-1-Reduktion). *Für jeden Zustand st gilt*

$$\text{reduce}(\text{reduce}(st)) = \text{reduce}(st). \quad (74)$$

Beweis. Dies ist `reduce_idem`. Im zweiten Reduktionsschritt verschwinden die Kreuzblöcke bereits, so dass das stabilisierte Schur-Komplement auf den vorhandenen Randblock kollabiert; genau dies ist `dtN_stabilized_fullMatrix_of_reduced`. $\square$

10.3 End-to-end-Spezialisierung auf den kanonischen Laplace-Zustand

Mit

$$st_{\text{Lap}} := \text{ofLaplacian}(L_0, h_{\text{det},0}) \quad (75)$$

werden die kanonischen Laplace-Daten in den aktiven Zustandsraum eingebettet. Dann beweist der Code:

$$(\text{reduce}(st_{\text{Lap}})).L_{BB} = \text{DtN}_{\text{stab}}^{\text{ToC}}(L_0), \quad (76)$$

also exakt `reduce_LBB_eq_dtn_stab`.

11 Semantische Identifikation von Stage 1

Das abstrakte Semantik-Interface `TailElimDtNStabilizedSemantics` fordert für ein gegebenes `TwoStageApprox`-Objekt eine volle Matrix $L(s)$, einen stabilisierten Inversenzeugen und ein Randbeobachtungsobjekt `boundaryOp`, so dass für alle Zustände s gilt:

$$\text{boundaryOp}(\text{stage1}(s)) = \text{DtN}_{\text{stab}}(L(s)). \quad (77)$$

Der ToC-spezifische Beweisdatensatz `TreeOfCliquesDtNStabilizedSemantics.mk` zeigt, dass der konkrete aktive Adapter exakt diese Semantik erfüllt.

Theorem 11.1 (Stage 1 ist stabilisierte DtN-Tail-Elimination). *Im aktiven ToC-Adapter ist Stage 1 mathematisch identisch mit einem stabilisierten DtN-/Kron-Eliminationsschritt.*

Beweis. Die Aussage ist der Kern von `TreeOfCliquesDtNStabilizedSemantics.mk` zusammen mit dem Feld `stage1_boundaryOp_eq` des abstrakten Interfaces. Da `stage1 = reduce` und `boundaryOp = LBB`, reduziert sich der Beweis auf das Ausklappen der Definition von `reduce`. $\square$

12 Stage 2 im konkreten ToC-Adapter

Im aktiven Adapter wird Stage 2 durch Maskierung des Randblocks und der Kreuzblöcke realisiert. Für eine Randregion $R \subseteq B_k$ gilt:

$$(\text{mask}_{BB}^R M)(i, j) := \begin{cases} M(i, j), & i, j \in R, \\ 0, & \text{sonst,} \end{cases} \quad (78)$$

$$(\text{mask}_{BI}^R M)(b, i) := \begin{cases} M(b, i), & b \in R, \\ 0, & \text{sonst,} \end{cases} \quad (79)$$

$$(\text{mask}_{IB}^R M)(i, b) := \begin{cases} M(i, b), & b \in R, \\ 0, & \text{sonst.} \end{cases} \quad (80)$$

Die konkrete Auswahlfunktion `select` lässt den Interior-Block unverändert und maskiert nur die sichtbaren Randteile. Genau dies ist die derzeit formalisierte Stage 2-Oberfläche. Aktuell wird sie auf dem strikten $A \rightarrow B$ -Pfad noch nicht zur Konstruktion von Pillar B benötigt; dort wird nur Stage 1 aktiv als Exportschritt benutzt.

13 Strikte $A \rightarrow B$ -Handoff-Fläche und Lean-Endobjekte

13.1 Allgemeine Handoff-Daten

`PillarA/Core/AQFTHandoff.lean` definiert die generische Übergabestruktur `AQFTHandoffData` als Zwölf-Felder-Kontrakt:

$$(\Omega, \text{instFintype}_\Omega, \text{instDecEq}_\Omega, RN, State, stage1, boundaryOp, cfgArity, cfgArity_pos, beta, beta_pos, phi). \quad (81)$$

Damit verlangt der Handoff formal nicht nur einen Boundary-Träger Ω , sondern auch die beiden Instanzfelder $\text{instFintype}_\Omega : \text{Fintype } \Omega$ und $\text{instDecEq}_\Omega : \text{DecidableEq } \Omega$. Ebenso sind $cfgArity_pos : 0 < cfgArity$ und $beta_pos : \forall s \ 0 < beta(s)$ keine nachträglichen Meta-Bemerkungen, sondern echte Strukturfelder und damit Teil der formalen Übergabedaten. Auf dem ToC-spezifischen aktiven Pfad werden die zugehörigen Komponenten durch `TreeOfCliquesAQFTHandoff` bzw. spätere konkrete Instanzen gefüllt.

13.2 Konkrete ToC-Exportobjekte

Die wichtigsten strikten Endobjekte lauten:

$$\Omega := B_{L_{\max}}, \quad (82)$$

$$st_0 := st_{\text{Lap}}, \quad (83)$$

$$st_1 := \text{reduce}(st_0), \quad (84)$$

$$L := (st_1).L_{BB}, \quad (85)$$

$$H_{\mathbb{C}}(i, j) := L(i, j) \in \mathbb{C}. \quad (86)$$

Der Code beweist dazu insbesondere:

- (i) Ω ist im nichtdegenerierten Regime $b > 0$ nichtleer (`omega_nonempty_of_pos_base`),
- (ii) $L^T = L$ (`L_transpose_eq`),

(iii) $H_{\mathbb{C}}$ ist hermitesch (`HC_isHermitian`).

Theorem 13.1 (Symmetrie des exportierten Randoperators). *Der aus Stage 1 exportierte reelle Randoperator erfüllt*

$$L^{\top} = L. \quad (87)$$

Beweis. Dies ist `L_transpose_eq`. Der Beweis benutzt: (a) die Symmetrie des ToC-Laplacians, (b) die blockweise Transpositionssätze des Splits, (c) die Symmetrie des stabilisierten Interior-Blocks und (d) die Invarianz der Inversenbildung unter Transposition für symmetrische invertierbare Matrizen. Danach wird das stabilisierte Schur-Komplement explizit transponiert und wieder auf sich selbst zurückgeführt. $\square$

Korollar 13.2 (Hermitizität der komplexifizierten Exportmatrix). *Die komplexifizierte Matrix $H_{\mathbb{C}}$ ist hermitesch.*

Beweis. Dies ist `HC_isHermitian` und folgt sofort aus der Reell-Symmetrie von L . $\square$

13.3 Weitere aus Pillar A exportierte diagnostische Größen

Zusätzlich exportiert `TreeOfCliquesAQFTHandoff` diagnostische und skalenbezogene Größen. Dabei muss strikt zwischen zwei verschiedenen β -Begriffen unterschieden werden:

$$\beta(p, st) := 1 + \text{signal}(p, st), \quad \text{signal}(p, st) := \|\text{boundaryOp}(st)\|_F, \quad (88)$$

ist die *state-abhängige* Größe aus `TreeOfCliquesAQFTHandoff.beta`; genau sie ist kompatibel mit dem Strukturfeld `AQFTHandoffData.beta : State  $\rightarrow$   $\mathbb{R}$` . Daneben existiert die rein policy-basierte diagnostische Größe

$$\text{axis}(p) := \text{kanonische scale-breaking axis}, \quad (89)$$

$$k_{\text{window}}(p) := K_{\max}(p) = \text{kWindow}(p), \quad (90)$$

$$k_*(p) := \text{kStar}(p) := \text{argmaxBreakUpTo}(\text{axis}(p))(k_{\text{window}}(p)), \quad (91)$$

$$\text{signalAt}(p, k) := \text{breakMeasure}(k)^{\text{toReal}}, \quad (92)$$

$$\text{signalFromA}(p) := \text{signalAt}(p, \text{kStar}(p)), \quad (93)$$

$$\beta_A(p) := \text{betaFromA}(p) := 1 + |\text{signalFromA}(p)|, \quad (94)$$

$$d_A(p) := \text{dFromA}(p) := K_{\max}(p) + 1, \quad (95)$$

$$\phi_A(p, i) := \text{phiFromA}(p, i) := \text{signalAt}(p, i.1). \quad (96)$$

Damit ist die Verbindung im Code explizit: `kStar` ist gerade der deterministisch aus `argmaxBreakUpTo` gewonnene Skalenindex, und `signalFromA` wird definitiv als `signalAt p (kStar p)` gebildet. Die Notation ist wichtig: `phiFromA` ist die konkrete policy-basierte Exportfunktion, während `phi` in `AQFTHandoffData` nur der generische Name des entsprechenden Strukturfeldes ist.

Theorem 13.3 (Positivität der exportierten inversen Temperatur). *Für jedes Policyobjekt p gilt*

$$\beta_A(p) > 0. \quad (97)$$

Beweis. Dies ist `betaFromA_pos`. Der Beweis ist elementar, da $|\text{signalFromA}(p)| \geq 0$ und somit $1 + |\text{signalFromA}(p)| \geq 1 > 0$. $\square$

Theorem 13.4 (Positivität der lokalen Konfigurationsarität). *Für jedes Policyobjekt gilt*

$$d_A(p) = K_{\max}(p) + 1 > 0. \quad (98)$$

Beweis. Dies ist `dFromA_pos` und folgt unmittelbar aus der Definition $d_A = K_{\max} + 1$. $\square$

14 Weitere abgeleitete Lean-Endobjekte auf Basis von Pillar A

Auch wenn nicht alle folgenden Objekte Teil des engsten strikten $A \rightarrow B$ -Kernpfades sind, sind sie kanonisch aus Pillar A oder aus dessen Exportflächen abgeleitet und damit reviewerrelevant:

- **REAL tower** aus stabilisierter DtN über alle Skalen: `Examples/TreeOfCliques_RealTower_FromDtNStabilized.lean`.
- **Scale-breaking axis** und Break-Witness-Objekte: `PillarA/Physics/Instances/TreeOfCliquesAxisCancelTreeOfCliquesBreakWitness.lean`.
- **$A \rightarrow C$ /OQS-Kompiler:** `Examples/TreeOfCliques_OQS_CompilerFromA.lean` und `TreeOfCliques_OQS_FromA.lean`.
- **$A \rightarrow$ Matter-Surface:** `Examples/TreeOfCliques_MatterFromA.lean`.
- **$A \rightarrow$ AQFT-Beispielmodule:** `Examples/TreeOfCliques_AQFT_Derived.lean`, `TreeOfCliques_AQFT_FromA.lean`, `TreeOfCliques_AQFT_HK_Instances.lean`.

Diese Module benutzen den aktiven Pillar-A-Export, sind aber ihrerseits bereits Übergänge in spätere Schichten oder Demo-/Instanziierungsoberflächen.

15 Belastbare Gesamtableitung des aktiven strikten Pfades

Die belastbare formale Ableitungskette des aktuell aktiven Pfades lässt sich damit kompakt so schreiben:

$$\begin{aligned}
 &(b, p : L_{\max}) \text{ in der Bedeutung } p : \text{Policy}(b) \implies \text{Addr}(b), \text{cellsAtLevel}, \text{cellsUpTo} \\
 &\implies V_k, B_k, I_k, e_k : V_k \cong B_k \sqcup I_k \\
 &\implies \text{Adj}, c_k, L_k = \Delta(c_k) \\
 &\implies L_0 := L_k, k = p.L_{\max} \\
 &\implies L_{BB}^0, L_{BI}^0, L_{IB}^0, L_{II}^0 \\
 &\implies \text{symm}(L_{II}^0), L_{II}^{\text{stab},0} \\
 &\implies \text{quadForm_L_nonneg} \implies \text{hpsd_laplacian_blockII} \implies \text{detGate_proved} \\
 &\implies \text{hpsd0_proved} \implies \text{hdet0_inst} \implies st_{\text{Lap}} \\
 &\implies st_1 = \text{reduce}(st_{\text{Lap}}) \\
 &\implies L = (st_1).L_{BB} = \text{DtN}_{\text{stab}}^{\text{ToC}}(L_0) \\
 &\implies L^T = L \implies H_{\mathbb{C}} = L \otimes_{\mathbb{R}} \mathbb{C} \text{ hermitesch} \\
 &\implies \Omega, \beta(p, \cdot), \beta_A(p), d_A(p), \phi_A = \text{phiFromA}, \text{Axis-/Signal-Exports und weitere A-Interfaces.}
 \end{aligned}$$

16 Schlussbemerkung

Der derzeit aktive mathematische Kern von Pillar A ist klar: ein diskret-hierarchisches ToC-Substrat, ein kanonisch daraus abgeleiteter Laplace-Operator, ein deterministisch stabilisierter DtN-/Kron-Schritt als Stage 1 und eine formalisierte, aber im aktuellen $A \rightarrow B$ -Kernpfad noch nicht aktiv genutzte Stage 2-Subsystemauswahl. Zum aktiven Kernpfad gehören dabei ausdrücklich auch das Umbrella-Modul `Ideal/TreeOfCliques/DtNStabilized.lean` sowie dessen Gate-Untermodule `DtNStabilizedGate.lean`; ohne diese ist der strikte Stabilitätsabschluss nicht vollständig beschrieben. Die Exportfläche in Richtung Pillar B basiert auf dem aus Stage 1 hervorgehenden Randoperator L , dessen Symmetrie und komplexe Hermitizität im Code bewiesen sind. Die historischen nicht-stabilisierten Pfade existieren noch, sind aber nicht der gegenwärtige aktive Kernpfad.

A Deklaratives Inventar der Pillar-A- und ToC-bezogenen Lean-Symbole

Die folgenden Tabellen listen die in `PillarA` sowie in den ToC-bezogenen Beispieldateien gefundenen Deklarationen auf. Jede Modulsektion beginnt zusätzlich mit einer knappen Funktionsbeschreibung und einer Statusmarkierung **strikt**, **historisch** oder **legacy**. Der Status bezieht sich auf die in diesem Dokument rekonstruierte heutige Codeverwendung: **strikt** bezeichnet den aktuellen formalen Kern bzw. dessen direkt benötigte Trägerschichten, **historisch** supersedierte frühere Pfade, und **legacy** beibehaltene Hook-, Interface-, Scaffold- oder Beispielschichten außerhalb des aktuellen strikten Kernpfads.

A.1 `PillarA/Core/AQFTHandoff.lean`

Kurzbeschreibung. Generische Struktur der $A \rightarrow B$ /AQFT-Handoff-Daten mitsamt Positivitätsgates für Arity und inverse Temperatur. *Status.* **strikt**.

Zeile	Art	Name
25	structure	<code>AQFTHandoffData</code>
48	theorem	<code>gate_cfgArity_pos</code>
51	theorem	<code>gate_beta_pos</code>

A.2 `PillarA/Core/Approximant.lean`

Kurzbeschreibung. Abstrakte endliche Approximationen mit Kardinalität sowie `refine/coarsen/full`-Operationen. Im aktuellen narrativ rekonstruierten Stage-1-Kernpfad wird dieses Modul nicht direkt benutzt. *Status.* **legacy**.

Zeile	Art	Name
22	structure	<code>Approximant</code>
33	def	<code>card</code>
36	def	<code>full</code>
40	def	<code>refine</code>
44	def	<code>coarsen</code>

A.3 PillarA/Core/BInterfaces/ChannelNet.lean

Kurzbeschreibung. Abstrakte B-Schnittstelle für kanalwertige Netze und ihre Funktorialität. *Status.* legacy.

Zeile	Art	Name
31	structure	ChannelNet
52	theorem	gate_CHAN_apply_id
56	theorem	gate_CHAN_apply_comp

A.4 PillarA/Core/BInterfaces/CovariantNets.lean

Kurzbeschreibung. Kovariante B-Schnittstellen für Algebren-, Zustands- und Kanalnetze unter Regionenaktionen. *Status.* legacy.

Zeile	Art	Name
29	structure	RegionAction
39	theorem	act_le
46	structure	CovariantLocalAlgebraNet
56	structure	CovariantStateNet
66	structure	CovariantChannelNet
87	theorem	gate_alg_naturality
94	theorem	gate_state_naturality
101	theorem	gate_channel_naturality

A.5 PillarA/Core/BInterfaces/GlobalCone.lean

Kurzbeschreibung. Abstrakte B-Schnittstelle für globale Konen und deren Restriktionskompatibilität. *Status.* legacy.

Zeile	Art	Name
22	structure	GlobalStateCone
31	theorem	gate_cone_compat
36	def	fromTop
53	theorem	gate_cone_fromTop_compat

A.6 PillarA/Core/BInterfaces/LocalAlgebraNet.lean

Kurzbeschreibung. Abstrakte B-Schnittstelle für lokale Algebranetze und Isotonie-/Kompatibilitätsgates. *Status.* legacy.

Zeile	Art	Name
30	structure	LocalAlgebraNet
46	theorem	gate_ALG_include_id
50	theorem	gate_ALG_include_comp

A.7 PillarA/Core/BInterfaces/RelativeEntropyHook.lean

Kurzbeschreibung. Hook-Schnittstelle für spätere relative-Entropie-Funktionale auf B-Seite. *Status.* legacy.

Zeile	Art	Name
35	structure	RelativeEntropyHook
45	theorem	gate_QRE_self
47	theorem	gate_QRE_DPI

A.8 PillarA/Core/BInterfaces/StateNet.lean

Kurzbeschreibung. Abstrakte B-Schnittstelle für zustandswertige Netze. *Status.* legacy.

Zeile	Art	Name
18	structure	here
30	structure	StateNet
46	theorem	gate_STATE_restrict_id
50	theorem	gate_STATE_restrict_comp

A.9 PillarA/Core/BoundaryPorts.lean

Kurzbeschreibung. Basistypen für Randports und deren endliche Indizierung. *Status.* strikt.

Zeile	Art	Name
35	structure	BoundaryPorts
50	def	mkAll
58	def	refine

A.10 PillarA/Core/CylinderSets.lean

Kurzbeschreibung. Zylinder-/Basis-Mengen auf endlichen Trunkationen des unendlichen Trägers. *Status.* legacy.

Zeile	Art	Name
38	def	cyl
45	theorem	cyl_mono
88	theorem	cyl_actRegion_preimage

A.11 PillarA/Core/Derived/GNS.lean

Kurzbeschreibung. Generische GNS-Hülle für spätere aus A exportierte Zustandsdaten. *Status.* legacy.

Zeile	Art	Name
28	structure	GNSData
40	class	HasDerivedGNS

A.12 PillarA/Core/Derived/GNS_CStar.lean

Kurzbeschreibung. C*-spezialisierte GNS-Verpackung als abgeleitete Interface-Schicht. *Status.* legacy.

Zeile	Art	Name
34	def	gnsCStarShim

A.13 PillarA/Core/Derived/BridgeToSubstrate.lean

Kurzbeschreibung. Brückenmodul von abgeleiteten GNS-/ModularKMS-Hooks zur späteren AQFTSubstrate-Oberfläche. *Status.* legacy.

Zeile	Art	Name
34	def	mkAQFTSubstrate_fromDerivedGNS
50	def	mkAQFTSubstrate_fromDerivedModularKMS

A.14 PillarA/Core/Derived/ModularKMS.lean

Kurzbeschreibung. Generische Platzhalter-/Interface-Schicht für modulare und KMS-bezogene Ableitungen. *Status.* legacy.

Zeile	Art	Name
27	class	HasDerivedModularKMS

A.15 PillarA/Core/Determinism.lean

Kurzbeschreibung. Determinismuskates für Update- und Exportschritte. *Status.* strikt.

Zeile	Art	Name
33	inductive	Level
48	structure	PolicyHash
59	theorem	gate_DO_DERIVED_PARAMS_OF_KEY
74	theorem	gate_POLICY_HASH_STABLE
87	structure	ExportBundle
95	structure	Exporter
102	class	ExportLaws
111	theorem	gate_D1_EXPORT_DEPENDS_ONLY_ON_KEY
118	def	ExportLaws

A.16 PillarA/Core/DirichletLaplacian.lean

Kurzbeschreibung. Abstrakte Dirichlet-/Laplacian-Schnittstelle für endliche Kerne. *Status.* strikt.

Zeile	Art	Name
39	def	offDiag
51	def	quadForm

Zeile	Art	Name
56	def	Sii
58	def	Sij
60	def	Sjj
62	lemma	degree_eq_sum_offDiag
66	lemma	Sii_eq_degree
77	lemma	offDiag_symm_of_symm
85	lemma	Sjj_eq_degree_of_symm
99	lemma	dirichletEnergy_expand
111	lemma	quadForm_laplacian_eq_degree_sub_Sij
129	theorem	gate_DIRICHLET_EQ_LAPLACIAN_QUAD
140	theorem	gate_DIRICHLET_NONNEG
179	def	clamp01
189	lemma	clamp01_nonexpansive
195	lemma	clamp01_sq_le
200	class	DirichletMarkovHook
205	theorem	gate_DIRICHLET_MARKOV_CLAMP01
214	def	DirichletMarkovHook

A.17 PillarA/Core/DirichletLaplacianBridge.lean

Kurzbeschreibung. Brückenlemmas zwischen abstrakten Dirichlet-Daten und Matrixdarstellungen. *Status.* strikt.

Zeile	Art	Name
10	abbrev	Weight

A.18 PillarA/Core/DynamicsEquivariant.lean

Kurzbeschreibung. Hook-Schicht für äquivariante Dynamiken auf Netzen und Zuständen. *Status.* legacy.

Zeile	Art	Name
31	abbrev	Dynamics
39	structure	EquivariantDynamics
60	theorem	gate_DYN_equivariant

A.19 PillarA/Core/Gates.lean

Kurzbeschreibung. Sammlung elementarer Gate-/Zeugenklassen für Positivität, Invertibilität und Strukturinvarianten. *Status.* strikt.

Zeile	Art	Name
45	abbrev	Weight
48	def	degree
51	def	laplacian
54	def	AdmissibleOp

Zeile	Art	Name
58	theorem	laplacian_is_symmetric_of_symm
75	theorem	laplacian_rowSum_zero
103	theorem	gate_LAPLACIAN_ADMISSIBLE
132	def	embedMatBV
135	def	FoldBV
137	theorem	gate_LAPLACIAN_FOLD
186	abbrev	TwoSidedInv
188	theorem	gate_DTN_WELLPOSED
212	theorem	gate_DTN_WELLPOSED_canonical
228	def	DtN
232	def	concatCols
238	def	concatRows
244	def	blockDiag
252	theorem	gate_GLUE_BW_IDENTITY

A.20 PillarA/Core/InfiniteCarrier.lean

Kurzbeschreibung. Abstrakte Klasse des unendlichen Trägers und seiner endlichen Kerne. *Status.* strikt.

Zeile	Art	Name
28	structure	InfiniteCarrier
52	abbrev	proj

A.21 PillarA/Core/InfiniteCarrierSymmetry.lean

Kurzbeschreibung. Symmetrie-Hilfssätze auf abstrakten unendlichen Trägern. *Status.* legacy.

Zeile	Art	Name
27	def	actInf
44	theorem	actInf_one
50	theorem	actInf_mul

A.22 PillarA/Core/MatrixNorms.lean

Kurzbeschreibung. Hilfssätze zu Matrixnormen und Frobenius-artigen Größen. *Status.* strikt.

Zeile	Art	Name
32	def	frobSq
35	def	frob
38	def	deltaSPD
40	theorem	deltaSPD_pos
53	theorem	frobSq_nonneg
62	theorem	frobSq_eq_zero_iff
96	theorem	frobSq_pos_of_ne_zero

A.23 PillarA/Core/NoetherHooks.lean

Kurzbeschreibung. Hook-Schicht für spätere Erhaltungssatz-/Noether-Verknüpfungen. *Status.* legacy.

Zeile	Art	Name
41	theorem	gate_COMMUTE_POW_LEFT
45	theorem	gate_COMMUTE_POW_RIGHT
59	structure	State
63	structure	Dynamics
71	def	conserved
80	theorem	gate_CONSERVED_OF_FIXED
106	structure	NoetherHook
115	theorem	gate_CHARGE_CONSERVED
126	def	mkByFixedness
145	structure	BalanceLaw
154	theorem	gate_BALANCE_SHAPE

A.24 PillarA/Core/Policy.lean

Kurzbeschreibung. Primitive Policy und deterministisch daraus abgeleitete Skalen-, Gewichtungs- und Regularisierungsdaten. *Status.* strikt.

Zeile	Art	Name
22	inductive	GroundRule
29	structure	Policy
42	abbrev	Key
45	def	key
47	theorem	key_congr
51	theorem	key_injective
60	def	canonical
63	theorem	canonical_eq_iff
76	def	L_int
79	def	K_max
81	theorem	K_max_def
84	def	groundRule
89	def	rStar
92	def	eta
99	def	epsMach64
102	def	epsFloor64
105	theorem	epsMach64_pos
110	theorem	epsFloor64_pos
115	def	eps
118	theorem	eps_pos
122	def	eps0
127	def	alphaDenom
137	def	alpha
143	theorem	alpha_eq_zero_of_not_mem

Zeile	Art	Name
147	theorem	alpha_eq_div_of_mem
158	theorem	gate_POLICY_derived_stable

A.25 PillarA/Core/RegionCore.lean

Kurzbeschreibung. Basistypen und Ordnungsstruktur für Regionen und Teilmengen. *Status.* strikt.

Zeile	Art	Name
30	abbrev	RegionAt
38	def	carrier
41	def	singleton
46	def	refineAt
50	def	coarsenAt

A.26 PillarA/Core/RegionNet.lean

Kurzbeschreibung. Generische Regionen-Netze über der Regionenkernstruktur. *Status.* strikt.

Zeile	Art	Name
41	structure	RegionNet
70	theorem	gate_REGION_le_refl
73	theorem	gate_REGION_le_trans
77	theorem	gate_REGION_le_antisymm
82	theorem	directed
89	theorem	gate_REGION_directed
93	theorem	le_top'
109	structure	RegionNetWithBoundary
123	theorem	gate_BOUNDARY_functorial
142	structure	RegionNetWithCompl
157	theorem	gate_COMPL_antitone
162	theorem	gate_COMPL_involutive
169	def	SetRegionNet
187	def	FinsetRegionNet
201	def	SetRegionNetWithCompl
257	structure	PreNet

A.27 PillarA/Core/ResistanceMetric.lean

Kurzbeschreibung. Abstrakte Widerstandsmetrik-Schnittstelle. *Status.* legacy.

Zeile	Art	Name
29	abbrev	RInf
32	structure	ResistanceMetric
45	theorem	gate_dist_self
49	theorem	gate_dist_symm
53	def	finiteR

Zeile	Art	Name
56	structure	ResistanceMetricLaws

A.28 PillarA/Core/ResistanceMetricCanonical.lean

Kurzbeschreibung. Kanonische/abgeleitete Widerstandsmetrik-Hülle. *Status.* legacy.

Zeile	Art	Name
44	abbrev	V
45	abbrev	I
53	def	ground
56	def	embedExcl
59	def	groundedMinor
65	def	symm
69	def	stabilized
72	def	rhs
77	def	dot
80	def	solves
86	def	delta
89	def	systemMatrix
102	def	gate_STABILIZED_INVERTIBLE
110	def	gate_RESISTANCE_SOLVE_WITNESS
121	def	gate_RESISTANCE_FAIL_TOP
126	def	gate_SUCCESS_SYMM
146	structure	ResistanceWellposedAxis
174	def	canonWeight
178	theorem	canonWeight_symm
183	theorem	canonWeight_nonneg
194	def	mkOf
226	def	mkNonvacuous
276	structure	CanonicalResistanceSpec
298	def	dist
301	theorem	gate_success_finite
309	theorem	gate_delta_pos

A.29 PillarA/Core/Selection.lean

Kurzbeschreibung. Stage-2-Subsystemmodi und Auswahloperatoren auf Trägerschichten. *Status.* strikt.

Zeile	Art	Name
32	structure	Selector
41	def	all

A.30 PillarA/Core/SubstrateClass.lean

Kurzbeschreibung. Abstrakte Substratklasse mit Kind-/Eltern- und Ebenenstruktur. *Status.* strikt.

Zeile	Art	Name
33	class	Substrate
47	def	ParentRel
51	def	ChildRel
55	def	refineSet
59	def	coarsenSet
63	theorem	refineSet_mono
70	theorem	coarsenSet_mono
83	class	DeterministicSubstrate
93	theorem	parent_mem_parents

A.31 PillarA/Core/SymmetryAction.lean

Kurzbeschreibung. Hook-Schicht für abstrakte Symmetriekaktionen. *Status.* legacy.

Zeile	Art	Name
20	structure	SymmetryAction
52	def	actRegion
57	def	actApproximant
62	def	actBoundaryPorts
73	def	SelectorEquivariant
84	structure	TailEliminationEquivariant
106	structure	UpdateKernelEquivariant

A.32 PillarA/Core/SymmetryCoverageGates.lean

Kurzbeschreibung. Hilfgates zur Abdeckung/Verträglichkeit von Symmetriekaktionen. *Status.* legacy.

Zeile	Art	Name
37	theorem	gate_infiniteCarrier_coh
41	theorem	gate_infiniteCarrier_ext
47	theorem	gate_infiniteCarrier_proj_def
56	theorem	gate_actInf_one
62	theorem	gate_actInf_mul
69	theorem	gate_actInf_commutes_with_proj
84	theorem	gate_covariant_alg_naturality
91	theorem	gate_covariant_state_naturality
98	theorem	gate_covariant_channel_naturality
115	theorem	gate_cone_compat
120	theorem	gate_cone_fromTop_compat
135	theorem	gate_equivariant_dynamics

A.33 PillarA/Core/TailEliminationCoherence.lean

Kurzbeschreibung. Kohärenzgesetze für Tail-Elimination und nachfolgende Auswahlsschritte. *Status.* strikt.

Zeile	Art	Name
40	class	TailProject
47	class	EBDProject
62	structure	TailEliminationCoherence
98	theorem	gate_TAIL_coherent_succ
143	theorem	gate_TAIL_ELIMINATION_EQUIVARIANT

A.34 PillarA/Core/TailInterface.lean

Kurzbeschreibung. Generische Zwei-Stufen-Schnittstelle mit truncateTail, integrateTail und select.
Status. strikt.

Zeile	Art	Name
34	class	TailInterface
41	abbrev	TailAt
49	class	TailEliminationHook
59	abbrev	EBD

A.35 PillarA/Core/ToC/AQFTSubstrate.lean

Kurzbeschreibung. AQFT-orientierte Sicht auf das ToC-Substrat als Träger lokaler Freiheitsgrade.
Status. strikt.

Zeile	Art	Name
38	structure	AQFTSubstrate

A.36 PillarA/Core/VacuumOffsetHook.lean

Kurzbeschreibung. Hook für spätere Vakuumoffset-/Energie-Normalisierungen. *Status.* legacy.

Zeile	Art	Name
21	class	VacuumOffsetHook

A.37 PillarA/Geometry/DimensionHooks.lean

Kurzbeschreibung. Hook-Schicht für Dimensions- oder effektive Geometrie Größen. *Status.* legacy.

Zeile	Art	Name
44	structure	HeatKernelHook
85	theorem	denom_ne_zero
95	theorem	dS_est_eq_zero_of_P_const
110	structure	SpectralDimensionHook
127	def	asHeat
130	theorem	gate_approx_const
142	def	LogLogLinear
151	theorem	dS_est_exact_of_loglog_zero

A.38 PillarA/Ideal/Adapter/InfiniteCarrierInstance.lean

Kurzbeschreibung. Konkrete ToC-Instanz der abstrakten InfiniteCarrier-Schnittstelle. *Status.* strikt.

Zeile	Art	Name
44	def	zeroDigit
47	def	canonicalAddr
55	theorem	take_replicate_succ
70	def	canonicalCell
88	def	canonicalCarrier
124	theorem	gate_actInf_available

A.39 PillarA/Ideal/Adapter/RegionNetEmbedding.lean

Kurzbeschreibung. Einbettung konkreter ToC-Regionen in das generische RegionNet-Interface. *Status.* strikt.

Zeile	Art	Name
25	def	cellRegion
28	theorem	cellRegion_antitone
36	theorem	cellRegion_child_le
43	theorem	boundary_child_le

A.40 PillarA/Ideal/Adapter/Seed.lean

Kurzbeschreibung. Konkrete Startdaten/Seeds für ToC-Adapterinstanzen. *Status.* strikt.

Zeile	Art	Name
24	abbrev	IdealSeedSpec

A.41 PillarA/Ideal/Adapter/SubstrateInstance.lean

Kurzbeschreibung. Konkrete ToC-Instanz der abstrakten SubstrateClass. *Status.* strikt.

Zeile	Art	Name
30	abbrev	IdealCell
40	def	parent
51	def	child

A.42 PillarA/Ideal/Adapter/TreeOfCliquesAQFTHandoff.lean

Kurzbeschreibung. Formaler Export vom ToC-Stage-1-Objekt zu AQFT-kompatiblen Handoff-Daten. *Status.* strikt.

Zeile	Art	Name
26	abbrev	Sem
30	abbrev	State

Zeile	Art	Name
34	def	signal
38	def	beta
41	theorem	signal_nonneg
45	theorem	beta_pos
51	theorem	beta_nonneg
57	abbrev	Ω
67	theorem	omega_nonempty_of_pos_base
74	theorem	omega_nonempty_of_Lmax_eq_zero
89	abbrev	st0
93	abbrev	st1
97	abbrev	L
184	theorem	L_transpose_eq
280	def	HC
289	theorem	HC_isHermitian
302	abbrev	axis
310	def	argmaxBreakUpTo
319	def	kWindow
322	def	kStar
326	def	signalAt
330	def	signalFromA
334	def	betaFromA
337	theorem	betaFromA_pos
348	def	dFromA
351	theorem	dFromA_pos
355	def	phiFromA

A.43 PillarA/Ideal/Adapter/TreeOfCliquesAllCellsConsistency.lean

Kurzbeschreibung. Konsistenzsätze zwischen allCells-Auswahl und konkreten Pipeline-Auswertungen. *Status.* strikt.

Zeile	Art	Name
35	abbrev	A
37	abbrev	Cell
38	abbrev	State
41	def	Obs
70	lemma	maskBB_diag
77	lemma	pipeline_allCells_eq
83	lemma	pipeline_single_LBB

A.44 PillarA/Ideal/Adapter/TreeOfCliquesApprox.lean

Kurzbeschreibung. Älterer nicht-stablisierter ToC-Approximationsexport. *Status.* historisch.

Zeile	Art	Name
45	abbrev	k

Zeile	Art	Name
46	abbrev	V
47	abbrev	B
48	abbrev	I
50	abbrev	sSplit
60	structure	ReducedOpState
94	def	sumMatrix
104	def	fullMatrix
132	def	interiorInv
143	def	maskBB
148	def	maskBI
153	def	maskIB
158	def	select
174	lemma	dtn_fullMatrix_of_cross_zero
187	def	reduce
196	lemma	dtn_fullMatrix_of_reduced
203	lemma	reduce_idem
222	def	mk

A.45 PillarA/Ideal/Adapter/TreeOfCliquesApproxDtNStabilized.lean

Kurzbeschreibung. Aktiver stabilisierter ToC-Approximationsexport samt Gate-Abschluss. *Status.* strikt.

Zeile	Art	Name
45	abbrev	k
46	abbrev	B
47	abbrev	I
48	abbrev	V
49	abbrev	sSplit
52	abbrev	L0
56	theorem	gate_L0_admissible
69	abbrev	LBB0
70	abbrev	LII0
71	abbrev	LBI0
72	abbrev	LIB0
89	abbrev	LII_stab0
94	abbrev	hdet0
97	abbrev	hpsd0
115	abbrev	c0
118	lemma	c0_symm
122	lemma	c0_nonneg
126	lemma	quadForm_L0_nonneg
141	lemma	symm_LII0_eq
158	lemma	quadForm_ext_eq_quadForm_blockII
304	theorem	hpsd0_proved
353	theorem	hdet0_of_hpsd0
364	def	hdet0_inst

Zeile	Art	Name
370	lemma	hdet0_to_stateHdet
380	structure	ReducedOpState
433	def	ofMatrix
479	def	ofLaplacian
492	def	ofLaplacian_of_posSemidef
496	def	maskBB
500	def	maskBI
504	def	maskIB
514	def	sumMatrix
522	def	fullMatrix
560	lemma	fullMatrix_ofMatrix
623	lemma	fullMatrix_ofLaplacian
643	lemma	hdet_LII_stabilized_fullMatrix
651	def	invStab
668	def	select
687	lemma	dtn_stabilized_fullMatrix_of_cross_zero
701	def	reduce
709	lemma	dtn_stabilized_fullMatrix_of_reduced
717	lemma	reduce_idem
744	def	stLap
747	lemma	fullMatrix_stLap
752	lemma	reduce_cross_zero_stLap
761	lemma	reduce_LBB_eq_dtn_stab
795	def	mk

A.46 PillarA/Ideal/Adapter/TreeOfCliquesDtNSemantics.lean

Kurzbeschreibung. Ältere rohe DtN-Semantik für den ToC-Approximationsexport. *Status.* historisch.

Zeile	Art	Name
35	abbrev	A

A.47 PillarA/Ideal/Adapter/TreeOfCliquesDtNSTabilizedSemantics.lean

Kurzbeschreibung. Aktive stabilisierte DtN-Semantik des ToC-Exports. *Status.* strikt.

Zeile	Art	Name
35	abbrev	A
38	abbrev	V
40	abbrev	split
43	def	L
52	def	boundaryOp
55	def	mk

A.48 PillarA/Ideal/Foliation.lean

Kurzbeschreibung. Retainte Hook-Schicht für spätere Foliationsbegriffe. *Status.* legacy.

Zeile	Art	Name
25	lemma	eq_of_prefix_of_length_eq

A.49 PillarA/Ideal/IdealSymmetryNoether.lean

Kurzbeschreibung. Retainte Noether-/Symmetrie-Verknüpfung auf idealisierter Schicht. *Status.* legacy.

Zeile	Art	Name
38	abbrev	DigitPerm
41	def	actAddr
55	def	addrEquiv
66	theorem	gate_actAddr_preserves_prefix
76	abbrev	LevelWord
79	def	actWord
82	def	wordToAddr
85	theorem	gate_wordToAddr_equivariant
100	def	actIdealCell
122	theorem	actIdealCell_leftInv
129	theorem	actIdealCell_rightInv
136	theorem	actIdealCell_bijective
141	lemma	actAddr_take
153	lemma	parent_equivariant
160	lemma	actAddr_child
164	lemma	child_equivariant
296	def	WeightInvariant
299	def	pull
309	def	permMatrix
312	theorem	gate_DIRICHLET_invariant_under_perm
398	theorem	gate_LAPLACIAN_QUAD_invariant_under_perm
525	theorem	gate_NOETHER_COMMUTE_LAPLACIAN

A.50 PillarA/Ideal/LCPMeasure.lean

Kurzbeschreibung. Hilfsgrößen zur Messung lokaler Clique-/Pfadcharakteristika. *Status.* legacy.

Zeile	Art	Name
34	def	lcp
41	theorem	gate_lcp_prefix_left
61	theorem	gate_lcp_prefix_right
78	theorem	gate_lcp_comm
97	theorem	gate_lcp_length_le_left
101	theorem	gate_lcp_length_le_right
105	theorem	gate_lcp_maximal

Zeile	Art	Name
130	structure	Edge
135	def	owner
137	theorem	gate_owner_prefix_u
140	theorem	gate_owner_prefix_v
144	def	Edge
147	theorem	gate_owner_swap
151	theorem	gate_owner_iff_commonPrefix

A.51 PillarA/Ideal/SpacetimePaths.lean

Kurzbeschreibung. Retainte Hook-Schicht für spätere Raumzeitpfade. *Status.* legacy.

Zeile	Art	Name
35	def	Sigma
44	def	AddrStep
47	def	AddrSpacetime
56	theorem	gate_addrSpacetime_mem_slice_iff
64	abbrev	AddrWorldline
66	abbrev	AddrWorldTube
69	def	IsFoliationAdapted
72	def	IsCausalChain
75	def	IsFoliationAdaptedTube
79	def	IsCausalTube
82	theorem	gate_adaptedTube_mem_sigma
90	theorem	gate_adapted_mem_sigma
97	theorem	gate_causal_tau_mono
115	theorem	gate_precedes_tau_le
140	theorem	gate_precedes_len_succ_implies_child
171	def	toSTWorldline

A.52 PillarA/Ideal/Substrate.lean

Kurzbeschreibung. Konkretes IDEAL-Adresssubstrat des Tree of Cliques. *Status.* strikt.

Zeile	Art	Name
28	abbrev	Addr
30	def	level
32	def	child
34	def	children
37	def	cellsAtLevel
40	def	cellsUpTo
44	theorem	mem_cellsAtLevel_iff_length
62	theorem	mem_children_length
85	theorem	children_subset_next_level
105	theorem	card_cellsAtLevel

A.53 PillarA/Ideal/TreeOfCliques/Boundary.lean

Kurzbeschreibung. Definition der Randzellen und Boundary-Auswahl im ToC. *Status.* strikt.

Zeile	Art	Name
10	abbrev	Boundary
13	theorem	cellsAtLevel_subset_cellsUpTo
28	def	boundaryEmbedding
35	def	interiorSet
39	abbrev	Interior
42	def	interiorEmbedding

A.54 PillarA/Ideal/TreeOfCliques/CoarsenBoundary.lean

Kurzbeschreibung. Hilfssätze zur Randverträglichkeit unter Koarsenierung. *Status.* strikt.

Zeile	Art	Name
31	abbrev	Boundary
54	def	mkChild
68	theorem	mkChild_injective
82	def	fiber
86	theorem	mem_fiber_iff
91	theorem	fiber_eq_image_mkChild
119	theorem	fiber_card
154	theorem	coarsenMat_refineMat
235	theorem	coarsenMat_idealTower

A.55 PillarA/Ideal/TreeOfCliques/DtN.lean

Kurzbeschreibung. Älterer ToC-spezifischer roher DtN-Pfad ohne Stabilisierung. *Status.* historisch.

Zeile	Art	Name
37	abbrev	V
38	abbrev	B
39	abbrev	I
41	abbrev	s
43	abbrev	L
48	abbrev	LII
52	def	interiorInv_of_det
65	def	dtn_of_det

A.56 PillarA/Ideal/TreeOfCliques/DtNStabilized.lean

Kurzbeschreibung. Umbrella-Re-Export des aktiven stabilisierten ToC-DtN-Pfads. *Status.* strikt. Dieses Umbrella-Modul enthält absichtlich keine eigenen Deklarationen. Es re-exportiert jedoch DtNStabilizedDefs.lean und DtNStabilizedGate.lean und ist deshalb auf dem aktiven strikten Pfad relevant, obwohl hier keine separate Deklarationstabelle folgt.

A.57 PillarA/Ideal/TreeOfCliques/DtNStabilizedDefs.lean

Kurzbeschreibung. Definitionen des aktiven stabilisierten ToC-DtN-Pfads. *Status.* strikt.

Zeile	Art	Name
32	abbrev	s
35	abbrev	L
43	abbrev	LII_stab
49	def	dtn_stabilized_of_det

A.58 PillarA/Ideal/TreeOfCliques/DtNStabilizedGate.lean

Kurzbeschreibung. Gate-/Positivitätsabschluss des aktiven stabilisierten ToC-DtN-Pfads. *Status.* strikt.

Zeile	Art	Name
39	abbrev	DetGate
46	abbrev	c
49	lemma	c_symm
53	lemma	c_nonneg
57	lemma	quadForm_L_nonneg
77	abbrev	sSplit
79	lemma	quadForm_ext_eq_quadForm_blockII
242	lemma	symm_blockII_eq
269	theorem	hpsd_laplacian_blockII
319	theorem	detGate_proved
333	def	dtn_stabilized

A.59 PillarA/Ideal/TreeOfCliques/Edges.lean

Kurzbeschreibung. Kantenrelationen und Nachbarschaften im ToC. *Status.* strikt.

Zeile	Art	Name
10	def	parentRel
14	def	parentEdge
18	def	siblingEdge
22	def	adjAddr
25	theorem	parentEdge_symm
29	theorem	siblingEdge_symm
39	theorem	adjAddr_symm
56	def	Adj
59	theorem	Adj_symm
63	def	conductance
67	theorem	conductance_symm
78	theorem	conductance_nonneg

A.60 PillarA/Ideal/TreeOfCliques/Laplacian.lean

Kurzbeschreibung. Kanonischer ToC-Laplaceoperator auf endlichen Trunkationen. *Status.* strikt.

Zeile	Art	Name
17	def	laplacian
23	theorem	gate_LAPLACIAN_ADMISSIBLE_treeOfCliques

A.61 PillarA/Ideal/TreeOfCliques/Level1Facts.lean

Kurzbeschreibung. Elementare ToC-Basisfakten über erste Ebenen und Nichtleerheit. *Status.* strikt.

Zeile	Art	Name
24	theorem	boundary1_val_eq_singleton
41	theorem	fiber_rootB0_eq_univ

A.62 PillarA/Ideal/TreeOfCliques/SmallScales.lean

Kurzbeschreibung. Hilfssätze zu kleinen Skalen und Basiskonfigurationen. *Status.* strikt.

Zeile	Art	Name
10	def	rootAddr
15	def	rootB0
22	def	rootI1
46	theorem	interiorSet_zero
82	theorem	card_boundary1

A.63 PillarA/Ideal/TreeOfCliques/Split.lean

Kurzbeschreibung. System/Umwelt-Split in Boundary- und Interior-Teile. *Status.* strikt.

Zeile	Art	Name
29	abbrev	V
30	abbrev	B
31	abbrev	I
34	def	isBoundary
50	def	toSum
57	def	fromSum
62	def	equivVertexSum
116	def	split

A.64 PillarA/Ideal/TreeOfCliques/Vertex.lean

Kurzbeschreibung. Definition der ToC-Zellen/Vertices. *Status.* strikt.

Zeile	Art	Name
15	abbrev	Vertex
20	def	addr
23	def	depth

Zeile	Art	Name
26	theorem	mem_cellsUpTo_length_le
54	theorem	depth_le
58	def	castSucc
65	theorem	cellsUpTo_subset_succ

A.65 PillarA/Kinematics/Causality.lean

Kurzbeschreibung. Kinematik-Hook für Kausalitätsbegriffe. *Status.* legacy.

Zeile	Art	Name
29	structure	CausalSpeedLimit
41	def	WorldlineBounded
44	theorem	gate_worldline_bounded

A.66 PillarA/Kinematics/Clock.lean

Kurzbeschreibung. Kinematik-Hook für diskrete Uhren-/Zeitmarken. *Status.* legacy.

Zeile	Art	Name
34	structure	Clock
45	def	run
50	def	reading
53	def	readingST
60	structure	Operationalizes
69	theorem	gate_reading_eq_properTimeUpTo
83	theorem	gate_readingST_eq_properTimeUpTo
98	theorem	gate_accumulator_operationalizes
106	theorem	theorem_CLOCK_accumulator_reads_properTimeUpTo

A.67 PillarA/Kinematics/DiscreteSpacetime.lean

Kurzbeschreibung. Retainte diskrete Raumzeit-Hülle. *Status.* legacy.

Zeile	Art	Name
29	structure	DiscreteSpacetime
39	def	Sigma
42	structure	Worldline
47	structure	WorldTube
55	def	Centerline
58	def	IsCausalCenterline
62	def	gate_centerline_is_worldline
69	theorem	gate_tube_subset_slice
78	def	mkEmpty

A.68 PillarA/Kinematics/FrameChange.lean

Kurzbeschreibung. Kinematik-Hook für Rahmenwechsel. *Status.* legacy.

Zeile	Art	Name
22	abbrev	Evo
25	abbrev	Frame
28	def	timeIndependent
31	def	frameChange
34	def	relChange
37	theorem	gate_FRAMECHANGE_CONJ_CONST
44	theorem	gate_RELCHANGE_ID_OF_CONST
51	theorem	gate_NONINERTIAL_IF_RELCHANGE_NE_ID

A.69 PillarA/Kinematics/FromUpdate.lean

Kurzbeschreibung. Kinematik-Hook zur Ableitung aus Update-Schritten. *Status.* legacy.

Zeile	Art	Name
36	abbrev	Event
51	def	eventEmbed

A.70 PillarA/Kinematics/KinematicsHooks.lean

Kurzbeschreibung. Sammelmodul für kinematische Hooks. *Status.* legacy.

Zeile	Art	Name
23	abbrev	VelHist
25	def	hasPointwiseRestFrame
27	def	hasGlobalRestFrame
29	theorem	gate_GLOBAL_REST_IMPLIES_POINTWISE
43	def	nonInertial
45	theorem	gate_NONINERTIAL_IMP_NOT_TIMEINDEP

A.71 PillarA/Kinematics/ProperTime.lean

Kurzbeschreibung. Kinematik-Hook für Eigenzeitbegriffe. *Status.* legacy.

Zeile	Art	Name
34	abbrev	StepCost
37	def	properTimeUpTo
41	theorem	gate_PROPERTIME_zero
46	theorem	gate_PROPERTIME_one
61	class	ProperTimeLaws
72	def	zeroCost

A.72 PillarA/Kinematics/ProperTimeFromMetric.lean

Kurzbeschreibung. Kinematik-Hook zur Eigenzeit aus Metrikdaten. *Status.* legacy.

Zeile	Art	Name
36	structure	ProperTimeMetricHook
59	def	properTimeOf
62	theorem	gate_properTimeOf_succ
69	theorem	gate_cost_defined_along_worldline
75	def	mkTopSpeedLimit

A.73 PillarA/Kinematics/SpacetimeRegions.lean

Kurzbeschreibung. Kinematik-Hook für Raumzeitregionen. *Status.* legacy.

Zeile	Art	Name
31	abbrev	SpacetimeRegionNet
35	def	sliceRegion
40	def	tubeRegion
43	theorem	gate_tubeRegion_subset_slice

A.74 PillarA/Kinematics/Worldline.lean

Kurzbeschreibung. Kinematik-Hook für Weltlinien. *Status.* legacy.

Zeile	Art	Name
22	abbrev	Worldline
25	abbrev	WorldTube
28	def	TubeNonempty
31	def	IsCenterline
47	theorem	gate_CENTERLINE_IS_WORLDLINE
61	class	WorldTubeChoiceAxis
84	theorem	gate_TUBE_HAS_CENTERLINE

A.75 PillarA/OQS/DtN.lean

Kurzbeschreibung. Roher DtN-/Schurkomplement-Schritt auf blockzerlegten Matrizen. *Status.* strikt.

Zeile	Art	Name
12	def	mkLambdaFVarsUsedOnly'
266	structure	InteriorInv
290	structure	InteriorWellposedAxis
313	def	mkWitnessAxis
323	def	DtN_of
330	def	interiorSolve
335	def	boundaryFlux
350	theorem	boundaryFlux_eq_DtN_mulVec

Zeile	Art	Name
428	theorem	blockII_mulVec_interiorSolve

A.76 PillarA/OQS/DtNStabilized.lean

Kurzbeschreibung. Aktiver stabilisierter DtN-/Kron-Schritt mit Symmetrisierung und SPD-Regularisierung. *Status.* **strikt**. Dieses Modul ist die eigentliche deklarative Oberfläche des stabilisierten DtN-Schritts; insbesondere enthält es sowohl die zeugenlose Definition `DtN_stabilized` als auch den zeugenbasierten Pfad `DtN_stabilized_of` und den Det-Gate-Konstruktor `dtn_stabilized_of_det`.

Zeile	Art	Name
51	def	symm
55	def	stabilize
60	def	deltaSPD
77	abbrev	deltaNum
90	lemma	deltaNum_pos
96	lemma	deltaNum_ge_eps
107	def	LII_stabilized
113	structure	InteriorInvStabilized
122	def	interiorInvStabilized_of_det
141	def	DtN_stabilized
155	def	DtN_stabilized_of
165	def	dtn_stabilized_of_det

A.77 PillarA/OQS/DtNStabilizedCriteria.lean

Kurzbeschreibung. Kriterien- und Zeugenhilfen für den stabilisierten DtN-Abschluss. *Status.* **strikt**.

Zeile	Art	Name
46	theorem	det_stabilize_ne_zero_of_posSemidef
81	theorem	det_LII_stabilized_ne_zero_of_posSemidef

A.78 PillarA/OQS/DtN_TailHook.lean

Kurzbeschreibung. Hook-Schicht zwischen DtN und generischen Tail-Elimination-Interfaces. *Status.* **legacy**.

Zeile	Art	Name
37	structure	DtNTailData
53	abbrev	DtNBoundaryData

A.79 PillarA/OQS/LiebRobinsonHook.lean

Kurzbeschreibung. Hook für spätere Lieb-Robinson-artige Schranken. *Status.* **legacy**.

Zeile	Art	Name
42	structure	Evolution
49	structure	CommutatorNorm
54	structure	Support
65	structure	LiebRobinsonBound
82	structure	LiebRobinsonHook
97	def	toCausalSpeedLimit
107	theorem	gate_worldline_bounded
114	theorem	finite_c_of_finite_v
124	theorem	derive_step_bound_from_LR

A.80 PillarA/OQS/SplitInvariants.lean

Kurzbeschreibung. Hilfssätze zu Invarianten des System/Umwelt-Splits. *Status.* legacy.

Zeile	Art	Name
37	structure	Split
46	def	covers
51	theorem	gate_mem_sys_not_mem_env
55	theorem	gate_mem_sys_not_mem_tail
59	theorem	gate_mem_env_not_mem_tail
70	def	uniqueOwner
75	theorem	gate_disjoint_implies_uniqueOwner
93	def	ownerTrichotomy
98	theorem	gate_covers_ownerTrichotomy

A.81 PillarA/OQS/SysEnv.lean

Kurzbeschreibung. Blockzerlegung in System-, Boundary- und Environment-Teile samt Masken. *Status.* strikt.

Zeile	Art	Name
31	structure	Split
51	def	reindex
53	def	blockBB
55	def	blockBI
57	def	blockIB
59	def	blockII
61	theorem	reindex_transpose
71	theorem	blockBB_transpose_of_symm
83	theorem	blockII_transpose_of_symm

A.82 PillarA/Physics/ScaleBreakingMismatchD.lean

Kurzbeschreibung. Dependent mismatch-basierte Scale-Breaking-Diagnostik für variierende Skalenfamilien; nicht Teil des engsten strikten Stage-1-Kernpfads. *Status.* legacy.

Zeile	Art	Name
35	def	breakMeasure_fromMismatch_onTowerD
51	def	breakMeasure_fromMismatchRealD
60	def	breakMeasure_fromMismatchIdealD
69	def	axis_fromMismatchD

A.83 PillarA/Physics/Instances/ScaleAxisTreeOfCliquesDtN.lean

Kurzbeschreibung. Ältere rohe DtN-Instanz der Scale-Breaking-Achse. *Status.* historisch.

Zeile	Art	Name
38	abbrev	A
41	abbrev	Sem

A.84 PillarA/Physics/Instances/TreeOfCliquesAxisCanonicalD.lean

Kurzbeschreibung. Aktive kanonische ToC-Instanz der Scale-Breaking-Achse. *Status.* strikt.

Zeile	Art	Name
44	abbrev	S
65	def	axis0

A.85 PillarA/Physics/Instances/TreeOfCliquesAxisCanonicalLawsD.lean

Kurzbeschreibung. Gesetze/Hilfssätze zur kanonischen ToC-Scale-Achse. *Status.* legacy.

Zeile	Art	Name
64	lemma	mulVec_delta
78	theorem	linOpOfMatrix_injective
114	theorem	normalizeTrace_const_one
134	theorem	normalizeTrace_const_one_scale_ne_zero
142	theorem	smul_cancel_matrix
158	theorem	smul_cancel_linOp

A.86 PillarA/Physics/Instances/TreeOfCliquesBoundaryK1Shape.lean

Kurzbeschreibung. Konkrete Form des K=1-Rands als Baustein des Exportoperators. *Status.* strikt.

Zeile	Art	Name
37	theorem	adj_boundary1_boundary_of_ne
61	theorem	laplacian_boundary_boundary_eq_neg_one
89	theorem	blockBB_k1_offdiag_eq_neg_one
105	theorem	sum_offdiag_blockBB_k1_eq_neg_sub_one
154	theorem	blockBB_k1_diag_eq_b
190	theorem	dtn_stabilized_k1_diag_eq

Zeile	Art	Name
240	theorem	dtn_stabilized_k1_diag_indep
268	theorem	dtn_stabilized_k1_no_diag_gap

A.87 PillarA/Physics/Instances/TreeOfCliquesBreakWitness.lean

Kurzbeschreibung. Break-Witness-Objekte zur Auswahl der ausgezeichneten Skala. *Status.* strikt.

Zeile	Art	Name
59	theorem	adj_boundary1_root
83	theorem	laplacian_boundary_root_eq_neg_one
120	theorem	laplacian_root_boundary_eq_neg_one
168	theorem	sum_vertex_k1_split
222	theorem	sum_row_LBB_eq_one
343	theorem	sum_all_LBB_eq_b
375	theorem	LII_root_root_eq_b
505	theorem	delta_pos_k1
525	theorem	LII_stab_root_root_eq_b_add_delta
579	theorem	LII_stab_inv_root_root_eq_inv_b_add_delta
648	theorem	schur_entry_const_k1
710	theorem	coarsen0_real1_entry_eq_sum
725	theorem	sum_all_real1_pos
881	theorem	coarsen0_real1_pos
892	theorem	coarsen0_real1_ne_zero
928	theorem	laplacian_k0_eq_zero
943	theorem	real0_eq_zero

A.88 PillarA/Physics/ScaleBreaking.lean

Kurzbeschreibung. Abstrakte Scale-Breaking-Funktionale und Achsenbegriffe. *Status.* strikt.

Zeile	Art	Name
27	def	constFamily
34	structure	ScaleBreakingAxis
53	def	scaleInvariant
56	def	breaksScale
59	def	breaksScaleByMeasure
62	def	breaksScaleByMeasureIdeal
65	def	coarsenFrom
67	theorem	gate_COARSENFROM_zero
71	theorem	gate_COARSENFROM_succ
80	class	Laws
88	theorem	gate_BREAKMEASURE_zero
92	theorem	gate_BREAKMEASURE_pos
101	def	Laws
113	structure	ScaleBreakingAxisD
127	def	scaleInvariantD

Zeile	Art	Name
130	def	breaksScaled
133	def	breaksScaleByMeasured
136	def	breaksScaleByMeasureIdealD
142	def	coarsenFromD
148	theorem	gate_COARSENFROMD_zero
152	theorem	gate_COARSENFROMD_succ
157	class	LawsD
163	theorem	gate_BREAKMEASURED_zero
167	theorem	gate_BREAKMEASURED_pos
178	def	toD
200	def	toConst

A.89 PillarA/Physics/ScaleBreakingFaithfulness.lean

Kurzbeschreibung. Retainte Faithfulness-Hooks für Scale-Breaking-Funktionale. *Status.* legacy.

Zeile	Art	Name
36	structure	ScaleBreakingFaithfulness

A.90 PillarA/Physics/ScaleBreakingFaithfulnessD.lean

Kurzbeschreibung. Retainte D-Variante der Faithfulness-Hooks. *Status.* legacy.

Zeile	Art	Name
41	theorem	gate_breakMeasure_fromMismatch_onTowerD_zero_of_observed_eq
86	structure	ScaleBreakingFaithfulnessD

A.91 PillarA/Physics/ScaleBreakingMismatch.lean

Kurzbeschreibung. Mismatch-Hülle für Scale-Breaking-Diagnostik. *Status.* legacy.

Zeile	Art	Name
40	abbrev	OpObserver
92	theorem	gate_breakMeasure_zero_of_observed_eq
123	def	Laws_fromMismatch

A.92 PillarA/RG/ScaleWindow.lean

Kurzbeschreibung. Hilfsschicht für Skalenfenster und RG-nahe Auswahlbereiche. *Status.* legacy.

Zeile	Art	Name
26	structure	ScaleWindow
35	def	In
38	def	inSucc
47	theorem	kmin_in

Zeile	Art	Name
51	theorem	kmax_in
55	def	length
58	def	epsAt
64	theorem	eps_le_epsSup
73	def	sub
80	theorem	in_of_in_sub
93	def	relax
100	theorem	epsSup_relax_ge
120	def	EpsMonotone
123	def	EpsBounded
126	theorem	epsBounded_of_epsSup

A.93 PillarA/Symmetry/ApproxIsometry.lean

Kurzbeschreibung. Retainte Symmetrie-Hook für approximative Isometrien. *Status.* legacy.

Zeile	Art	Name
28	abbrev	Dist
31	structure	ApproxIsometry
46	def	id
57	def	comp
89	def	OnWindow
92	theorem	upper_epsSup
102	theorem	lower_epsSup
112	theorem	onWindow_comp_closed
123	theorem	onWindow_comp_relax

A.94 PillarA/Symmetry/ApproxPoincare.lean

Kurzbeschreibung. Retainte Symmetrie-Hook für approximative Poincare-Strukturen. *Status.* legacy.

Zeile	Art	Name
31	structure	GroupLikeAction
45	theorem	act_congr
52	structure	ApproxPoincare
93	theorem	trans_upper
103	theorem	rot_upper
111	theorem	boost_upper
119	theorem	trans_comp_contract
135	theorem	approx_commute_contract

A.95 PillarA/Symmetry/PoincareLorentzSplit.lean

Kurzbeschreibung. Retainte Hook-Schicht für Lorentz-/Poincare-Splits. *Status.* legacy.

Zeile	Art	Name
15	structure	Poincare
23	def	actPoint
25	def	actVel
27	theorem	gate_displacement_depends_only_on_Lorentz
41	theorem	gate_stepVelocity_depends_only_on_Lorentz
55	structure	VelocityHook
70	def	stepVel
72	theorem	gate_stepVel_trans
76	theorem	gate_stepVel_rot
80	theorem	gate_stepVel_boost

A.96 PillarA/Update/ApproximationPipeline.lean

Kurzbeschreibung. Generische Pipeline zur zweistufigen Approximation und Tail-Elimination.

Status. strikt.

Zeile	Art	Name
36	inductive	SubsystemMode
45	def	single
62	structure	TwoStageApprox
83	abbrev	FinestCell
85	def	finestCells
89	def	stage1
91	def	pipeline
95	theorem	gate_STAGE1_DEF
98	theorem	gate_PIPELINE_DEF
104	theorem	gate_TRUNCATE_IDEMPOTENT
107	theorem	gate_INTEGRATE_IDEMPOTENT
110	theorem	gate_STAGE1_IDEMPOTENT
123	def	mkOfLaws
145	def	mkIdStage1
172	class	MonotoneLaws
177	def	MonotoneLaws
184	theorem	gate_PIPELINE_MONOTONE
208	structure	Observer
219	class	AllCellsConsistency
235	def	obs_congr
239	def	true_witness
243	def	false_witness
249	theorem	gate_ALLCELLS_POINTWISE
275	abbrev	Cell
291	def	ModeInvariant
299	class	Laws
313	theorem	gate_STAGE1_EQUIVARIANT
318	theorem	gate_PIPELINE_EQUIVARIANT
331	def	stage1KernelEquivariant

Zeile	Art	Name
343	def	pipelineKernelEquivariant

A.97 PillarA/Update/DeterministicExport.lean

Kurzbeschreibung. Deterministischer Export stabilisierter Approximationen in Endobjekte. *Status.* strikt.

Zeile	Art	Name
37	def	exportId
41	theorem	gate_EXPORTID_DEPENDS_ONLY_ON_KEY
53	structure	Plan
64	def	id
68	def	emit
74	theorem	gate_EXPORT_DEPENDS_ONLY_ON_KEY
83	def	mkPlanFromTwoStage
95	def	emitStage1
102	def	emitPipeline

A.98 PillarA/Update/MismatchDriver.lean

Kurzbeschreibung. Treiber für diagnostische Mismatch-Funktionale. *Status.* legacy.

Zeile	Art	Name
41	structure	UpdateFromMismatch
58	theorem	gate_ZERO_MISMATCH_STEP_ID

A.99 PillarA/Update/MismatchFunctional/NormDiff.lean

Kurzbeschreibung. Konkretes Normdifferenz-Mismatchfunktional. *Status.* legacy.

Zeile	Art	Name
37	def	delta
53	def	matrixOfLinOp
97	def	idProjector
122	theorem	mismatch_self
137	theorem	mismatch_pos_of_matrix_ne_zero
156	def	mk
164	def	mkId

A.100 PillarA/Update/MismatchFunctional.lean

Kurzbeschreibung. Abstrakte Hülle für Mismatch-Funktionale. *Status.* legacy.

Zeile	Art	Name
34	structure	LinOp

Zeile	Art	Name
52	theorem	ext
69	structure	NullModeProjector
75	def	projectOp
81	theorem	projectOp_add
86	theorem	projectOp_zero
94	def	traceNormFactor
98	def	normalizeTraceVec
105	def	mismatchVec
121	abbrev	NormalizedOp
140	structure	MismatchFunctional
194	def	normalizeTrace
203	theorem	normalizeTrace_denom_pos
212	theorem	normalizeTrace_congr

A.101 PillarA/Update/OpObserver.lean

Kurzbeschreibung. Beobachter-/Interface-Schicht für operatorwertige Update-Ausgaben. *Status.* legacy.

Zeile	Art	Name
31	def	constFamily
34	structure	OpObserver
38	structure	OpObserverD
44	def	toD
59	def	toConst
71	abbrev	OpObserverFam
76	def	ofD
80	def	toD

A.102 PillarA/Update/Seed.lean

Kurzbeschreibung. Seeds/Startdaten auf der Update-Schicht. *Status.* legacy.

Zeile	Art	Name
26	structure	SeedSpec
34	theorem	deterministic
39	def	StableUnder
41	theorem	stable_id
45	theorem	stable_comp

A.103 PillarA/Update/Step.lean

Kurzbeschreibung. Einzelschritt der deterministischen Update-Dynamik. *Status.* strikt.

Zeile	Art	Name
26	abbrev	Conductance
29	def	updateStep
35	theorem	updateStep_fixed
41	theorem	updateStep_dynamic
47	theorem	updateStep_preserves_nonneg

A.104 PillarA/Update/TailEliminationDtN.lean

Kurzbeschreibung. Älterer Tail-Elimination-Pfad über den rohen DtN-Schritt. *Status.* historisch.

Zeile	Art	Name
35	def	linOpOfMatrix
51	structure	TailElimDtNSemantics
89	def	opObserver
93	theorem	gate_STAGE1_IS_DTN

A.105 PillarA/Update/TailEliminationDtNSTabilized.lean

Kurzbeschreibung. Aktive Tail-Elimination über den stabilisierten DtN-Schritt. *Status.* strikt.

Zeile	Art	Name
32	structure	TailElimDtNSTabilizedSemantics

A.106 Examples/TreeOfCliques_AQFT_Derived.lean

Kurzbeschreibung. Beispielpfad für aus A abgeleitete AQFT-Objekte. *Status.* legacy.

Zeile	Art	Name
50	abbrev	LSAN_ToC
54	abbrev	SN_ToC
58	abbrev	Cone_ToC

A.107 Examples/TreeOfCliques_AQFT_FromA.lean

Kurzbeschreibung. Beispielpfad des AQFT-Handoffs aus ToC-Daten. *Status.* legacy.

Zeile	Art	Name
29	abbrev	L

A.108 Examples/TreeOfCliques_AQFT_HK_Instances.lean

Kurzbeschreibung. Beispielhafte HK-/AQFT-Instanzen auf Basis des ToC-Exports. *Status.* legacy.

Zeile	Art	Name
46	def	oneState

Zeile	Art	Name
52	def	trivialDyn
63	theorem	one_invariant
70	theorem	one_kmsStrip
89	def	oneKMSSState
118	def	splitWitness_trivial
131	theorem	splitProperty_trivial
144	def	repDataPUnit
149	def	spectrumWitnessForward
155	def	spectrumConditionPUnit
170	abbrev	N_ToC
174	theorem	splitProperty_ToC
186	def	kms_local_demo

A.109 Examples/TreeOfCliques_EndToEnd.lean

Kurzbeschreibung. End-to-end-Beispiel des älteren ToC-Pfads. *Status.* legacy.

Zeile	Art	Name
34	abbrev	A
38	abbrev	Sem
49	abbrev	State
52	abbrev	Boundary
55	abbrev	BoundaryOp
60	theorem	boundaryOp_eq_blockBB
65	theorem	stage1_boundaryOp_eq_DtN

A.110 Examples/TreeOfCliques_EndToEnd_StabilizedConcrete.lean

Kurzbeschreibung. Konkretes End-to-end-Beispiel des stabilisierten Pfads. *Status.* legacy.

Zeile	Art	Name
43	abbrev	st
46	abbrev	A
48	abbrev	st_stage1
57	theorem	fullMatrix_st
60	theorem	reduce_LBB_st

A.111 Examples/TreeOfCliques_MatterFromA.lean

Kurzbeschreibung. Beispielpfad für mögliche Matter-/Nebenexports aus A. *Status.* legacy.

Zeile	Art	Name
50	def	argmaxBreakUpTo
59	def	kWindow
80	def	dFromA
83	theorem	dFromA_pos

Zeile	Art	Name
103	theorem	betaFromA_pos
108	theorem	matterFromA_beta_pos
113	theorem	beta_pos

A.112 Examples/TreeOfCliques_OQS_CompilerFromA.lean

Kurzbeschreibung. Beispiel eines OQS-Compilerpfads aus A-Daten. *Status.* legacy.

Zeile	Art	Name
41	abbrev	Mat
87	theorem	compiledUnitary_unitary
112	def	compileKraus
119	def	compileChannel
133	theorem	compileChannel_isCP
140	theorem	compileChannel_isTP
147	def	compileChannel_stinespring
159	theorem	compileChannel_hasStinespring
166	def	iterateChannel
174	def	derivedSemigroup
191	def	derivedGenerator
199	theorem	derivedGenerator_generates
207	def	derivedProcess
216	theorem	derivedProcess_memoryLaw

A.113 Examples/TreeOfCliques_OQS_FromA.lean

Kurzbeschreibung. Beispiel eines OQS-Exports aus dem ToC-Handoff. *Status.* legacy.

Zeile	Art	Name
28	abbrev	st1AB
55	def	S_oqs_fromA
60	def	G_oqs_fromA
65	def	P_oqs_fromA
69	theorem	G_oqs_fromA_generates
75	theorem	P_oqs_fromA_memoryLaw

A.114 Examples/TreeOfCliques_OQS_FromCoarsen.lean

Kurzbeschreibung. Beispielpfad über Koarsenierung und OQS-Auswertung. *Status.* legacy.

Zeile	Art	Name
40	abbrev	Mat
42	abbrev	RectMat
44	abbrev	RectKrausOps
50	abbrev	rectKrausMap
56	abbrev	RectKrausTP

Zeile	Art	Name
62	abbrev	IsRectKrausCP
68	abbrev	rectKrausChannel
74	theorem	rectKrausChannel_isCP
84	def	ptraceKraus
90	def	ptraceMap
95	def	ptraceCPTP
99	theorem	ptraceCPTP_isCP
112	def	stinespring_ptrace
124	theorem	hasStinespring_ptrace

A.115 Examples/TreeOfCliques_RealTower_FromDtNStabilized.lean

Kurzbeschreibung. Beispielpfad vom stabilisierten DtN-Schritt zur endlichen Real-Tower. *Status.* legacy.

Zeile	Art	Name
31	abbrev	S

A.116 Examples/TreeOfCliques_ScaleAxisHarness.lean

Kurzbeschreibung. Beispiel-/Harness-Modul für die Scale-Axis-Diagnostik. *Status.* legacy.

Zeile	Art	Name
47	abbrev	S
75	def	axis0

A.117 Nicht separat inventarisierte Import-/Umbrella-Module

Die folgenden Lean-Dateien unter `PillarA` werden im vorstehenden Deklarationsinventar nicht mit eigener Tabelle aufgeführt, weil sie entweder reine Import-/Umbrella-Dateien ohne eigene Deklarationen sind oder im aktuellen Dokument nur als Sammel- bzw. Exportoberflächen fungieren. Für einen Reviewer ist damit explizit sichtbar, dass ihr Fehlen im Inventar *keine* versehentliche Auslassung fachlicher Kerndefinitionen bedeutet.

Datei	Einordnung
<code>PillarA/All.lean</code>	Umbrella/Imports
<code>PillarA/BuildAll.lean</code>	Umbrella/Imports
<code>PillarA/Core/Exports.lean</code>	Export-/Umbrella-Oberfläche
<code>PillarA/Core/Interfaces.lean</code>	Interface-/Umbrella-Oberfläche
<code>PillarA/Ideal/Adapter/Foliation.lean</code>	Import-/Nebenoberfläche
<code>PillarA/Ideal/Adapter/IdealSymmetryNoether.lean</code>	Import-/Nebenoberfläche

Datei	Einordnung
PillarA/Ideal/Adapter/LCPMeasure.lean	Import- /Nebenoberfläche
PillarA/Ideal/Adapter/SpacetimePaths.lean	Import- /Nebenoberfläche
PillarA/Ideal/Adapter/Substrate.lean	Umbrella/Imports
PillarA/Ideal/Adapter/TreeOfCliquesAll.lean	Umbrella/Imports
PillarA/Ideal/Adapter/UpdateInstancesAll.lean	Umbrella/Imports
PillarA/Ideal/TreeOfCliques/All.lean	Umbrella/Imports
PillarA/Interface.lean	Umbrella/Imports
PillarA/Kinematics/SpatialMetric.lean	Import- /Nebenoberfläche
PillarA/PillarA.lean	Umbrella/Imports
PillarA/Scaffold/All.lean	Umbrella/Imports
PillarA/Scaffold/Interfaces.lean	Umbrella/Imports
PillarA/Update/Instances/All.lean	Umbrella/Imports
PillarA/Update/Instances/TreeOfCliquesApprox.lean	Umbrella für histori- schen Pfad
PillarA/Update/Instances/TreeOfCliquesDtNSemantics.lean	Umbrella für histori- schen Pfad
PillarA/Update/TwoStageApprox.lean	historischer Re- Export